Python

→ python is high-level interpreted general purpose prgming language. It is known for its simple syntax and easily readable.

→ python supports both procedural oriented and object oriented prgming language.

→ python is widely used in many areas both for development and testing

Ex: Application testing, webbase testing and storage testing.

→ python is dynamically prgming lang, no need to declare a type of variable. python interpreter will take care of it in run time.

→ python has cross platform support and it is independent of system hardware.

→ python has large collection of in-built modules (reusable code) that makes simple for testers to write code- reuse the code.

→ syntax to execute python is python+ script name

→ How python works?

python code is executed through an interpreter the interpreter reads the python source code translates it in byte code and then the python virtual machine (PVM) executes this bytecode.

Python

→ python is high-level interpreted general purpose prgming language. It is known for its simple syntax and easily readable.

→ python supports both procedural oriented and object oriented prgming language.

→ python is widely used in many areas both for development and testing

Ex: Application testing, webbase testing and storage testing.

→ python is dynamically prgming lang, no need to declare a type of variable. python interpreter will take care of it in run time.

→ python has cross platform support and it is independent of system hardware.

→ python has large collection of in-built modules (reusable code) that makes simple for testers to write code- reuse the code.

→ syntax to execute python is python+ script name

→ How python works?

python code is executed through an interpreter the interpreter reads the python source code translates it in byte code and then the python virtual machine (PVM) executes this bytecode.

Difference between compiler and Interpreter ?

Compiler	Interpreter
→ Translates entire prgm into machine code before execution	→ Translates and executes code line-by-line
→ shows all errors after compiling	→ shows errors one at a-time

Interpreter:

Source code → compiler → Bytecode → virtual machine → Running machine code → output

→ python is slower (interpreter language) compare to C, C++, java (compiler language)

→ still we are going for python because of simple syntax, easy readability, large community support

→ Development speed > execution speed

PVM: It is a runtime engine that executes python bytecode. It acts like a bridge b/w python code & system hardware. It reads bytecode instructions and execute them line by line.

Mutable: The object can be changed after it is created

immutable: " " cannot " " once created

→ ① Interactive mode ② Scripting mode

Topics

1. Basics of python
   - → Intro of python
   - → How python works
   - → Installing python
   - → comments in python
2. Data types
   - → list str, tuple, dict, set, int, Bool e.t.c
3. Operators
4. loops and conditions
   - → for, while & if
   - → loop control statements
5. functions
   - → Basics of functions, function arguments
   - → lamba function, filter, map and reduce
6. modules and packages
   - → creating and reusing modules & packages
   - → inbuilt modules
   - → user defined modules
   - → External modules
7. file handling and regular expressions
8. Exception / Error Handling
9. oops
   - → Introduction to oops
   - → features of oops
10. Advanced topics
    - → Decorators
    - → Generators
    - → Iterators

→ comprehensions (list & dict)

11. standing and writing automation (pylint & peps)

*12. Test case automation (test framework)
(pytest & unit test)

13. version control system (git & BitBucket)

Data types

⇒ Data types in python is used to store the data we need to assign a data to a variable.

Ex: x = 100
    name = "python"

In python there are different data types. They are:

→ int
→ string
→ float
→ boolean
→ list
→ tuple
→ set
→ dictionery

int : int stores whole number

Ex: x = 100
    Y = 0

To get ~~dat~~ type of data : type()

Syntax : type(variable_name)

Ex: type(int1) # class <int>

comments in python

used to comment [Exclude the lines of code from execution]

→ we can use "#" to comment a single line in python

Ex: # x = 100

→ Multiline commenting we use """ (triple quotes)

for ex: """
line 1
line 2
"""

→ dir() function: It is used to print all supported methods (operations) that we perform on variables.

Str: (Strings)

→ String is a stream of characters enclosed with quotes

→ Strings are immutable data types, once we define strings we cannot modify the strings.

→ Ex: str1 = "python"
str2 = "a"
str3 = 'python'

Strings can be declared in single (or) double quotes

→ # multiline strings

str4 = """ line 1
line 2
line 3"""

len(): It is used to get total number of characters in a string.

Syntax: len(str1)

Ex: str1 = "python"
print(len(str1))

O/P: 6

String methods

→
```
Str1 = "pythoN 3.0"
print (Str1.upper())     # PYTHON 3.0
print (Str1.lower())     # python 3.0
print (Str1.swapcase())  # pYTHON 3.0
print (Str1.captalize()) # Python
print (Str1.title())     # Python 3.0
```

upper: It will convert all in uppercases

lower: " " " " " lowercases

swapcase: If it is uppercase it converts to lowercase

" " " lowercase " " " uppercase

captalize: first letter it will convert to uppercase

title: first letter of each word it will convert to uppercase

Ex: python language 3.0

Python Language 3.0

Index: → Index in python used to find the position of a character in strings.

→ Index always starts from zero

→ we can access the individual elements by its index value

Ex:
```
Str1 = "python"
       012345
print (Str1[0])  # P
```

There are two types of index

① positive index: starts from 0.... N [start of strings]

② Negative index: starts from -1....-N [End of strings]

Ex: Str1 = "python"
-6 -5 -4 -3 -2 -1
Print (Str1[-1]) # n
Print (Str1[-6]) # P

find: It will find the first position of a character in the string

Ex: Str1 = "python version 3.13"
print(str1.find("python")) # 0

→ If there is non existing character it will print "-1"
→ Spaces are also consider in the string

print(Str1.find("zzz")) # -1
print(Str1.find("o")) # 4

Replace: It will replace one keyword with another keyword

Ex: Str1 = "python version 3.0"
Str2 = Str1.replace("python", "linux")
print(Str2) # linux version 3.0

Slicing in python: used to extract some portion from original string

syntax: str1[start_index:end_index]

Ex: Str1 = "python version 3.0"
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
print(Str1[0:6]) # python
print(Str1[3:6]) # ho
print(Str1.index("n")) # 5
print(Str1.index("3")) # 15

→ slicing works based on index values

index() method: used to get index of a given

character

⇒ Difference b/w find and index :

In index if string is not found it throws the error. In find if string is not found it writes "-1".

more examples:

```
str1 = "python version 3.0"
print (str1[:5]) # pytho
print (str1[0:]) # python version 3.0
print (str1[:]) # python version 3.0
print (str1[0:-1]) # python version 3.0
print (str1[-3:-1]) # 3.
```

step slicing: how many steps to skip

syntax: str1[start:end:step]

ex: str1 = "python"
0 1 2 3 4 5

```
print (str1[0:6:1])
0   # p
0+1 = 1 # y
1+1 = 2 # t
2+1 = 3 # h
3+1 = 4 # o
4+1 = 5 # n
```

```
print (str1[0:6:2])
0   # p
0+2 = 2 # t
2+2 = 4 # o
4+2 = 6 #
```

python
0 1 2 3 4 5
0 = 0 p
0+2 = 2 t
2+2 = 4 o
4+2 = 6

```
print (str1[:6:2]) # pto
print (str1[::2]) # pto
```

~~print (2:-1)~~

→ move backward by 1 step >> reverse the string

```
print (str1[::-1])
```

⟹ print all odd index characters
⟹ print all even index characters

```
Str1 = "abcabcabc"
       012345678
print (Str1[0:9:2]) # 02468 # acbac # ~~odd~~ even
print (Str1[1:9:2]) # 1357  # bacb  # even odd
```

String methods

→
```
Str1 = "python version 3.0"
print (str1.startswith ("p")) # True
print (Str1.startswith ("y")) # false
print (Str1.startswith ("python")) # True
```

⟹ startswith: Returns True if string starts with given substring else returs false

⟹ ends with ; Returns True if string ends with given substring else returns false

Ex:
```
Str1 = "file1.txt"
Str2 = "file2.py"
print (Str1.endswith (".py")) # false
print (str1.endswith (".txt")) # True
```

⟹ isdigit: It will print True if given string is all digits.

Ex:
```
Str1 = "9999abc"
print (str1.isdigit()) # False
```

~~Prin~~

→
```
Str1 = "999"
print (Str1.isdigit()) # True
```

⟹ isspace: It will print True if given string starts with space

Ex:
```
Str1 = "   "
print (Str1.isspace()) # True
```

⇒ isalpha: It will print True if given string is characters

Ex: Str1 = "abcabc"
Print (Str1. isalpha()) # True

⇒ isalnum: It will print True if given string is alphabets & characters

⇒ islower, isupper

Ex: Str1 = "abc"
Print (Str1. islower()) # True
print (Str1. isupper()) # False

⇒ strip(): It is used to remove leading and trailing whitespaces.

Ex: Str1 = " python 3.0 "
Str1 = " python 3.0 "
1 2 3 4 5 6 7 8 9 10 11 12
print (len (Str1)) # 12
Str1 = Str1. strip()
print (len (Str1)) # 10

⇒ rstrip(): Remove spaces from right

Ex: Str1 = " python 3.0 "
print (len (Str1)) # 12
Str2 = Str1. rstrip()
print (len (Str2)) # 11

⇒ lstrip(): Remove spaces from left

Ex: Str1 = " python 3.0 "
print (len (Str1)) # 12
Str2 = Str1. lstrip()
print (len (Str2)) # 11

⇒ rfind: It will find the last position of a character in the string.

Ex: str1 = "abcabc"
0 1 2 3 4 5
print (str1.rfind("a")) # 3
print (str1.find("a")) # 0

List

→ List is a collection of elements in python which should be declared in square bracets "[ ]"

→ List allows us to store multiple elements in list. Each element should be seperated with ","

→ List is mutable and ordered data type and it allows duplicate elements

→ we can store any type of data in list.

Ex: [1, 1.34, "str1", "python", "a"]

→ List supports indexing and slicing (+ve & -ve indexes)

→ List supports in-build functions like len(), type(), max(), min(), etc...

→ List take more memory

Ex: a=10
b=200
d=20
list1 = [a, b, "c", d]
print (len(list1)) # 4

Ex: list1 = [1, 2, 3, 4, 5]
print (list1[0]) # 1
print (list1[-1]) # 5
print (list1[1:3]) # [2,3]
print (list1[3:5]) # [4,5]
print (list1[2]) # 3

Ex:
```
list1 = [0, "1.2", 1.3, "a", "Python"]
print (type (list1 [1])) # <class 'str'>
print (len (list1 [1])) # 3
print (len (list1 [2])) # Type Error
print (list1 [-1][0]) # P
print (list1 [-1][-1]) # n
print (list1 [-1][0:3]) # Pyt
```

List methods:

→ append : To add single element at end of list

Syntax : list.append (element)

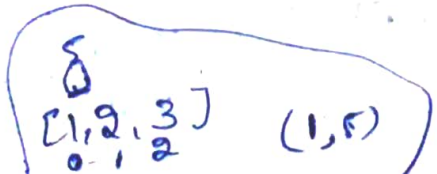
→ extend: To add multiple elements at end of list

syntax: list.extend ([element1, element2, --elementn])

→ insert: To add element at given position

Syntax: list.insert (pos, element)

Ex:
```
→ list1 = [1, 2, 3]
print (list1.append (4)) # [1, 2, 3, 4]
print (list1)
print (list1.append (4)) # [1, 2, 3, 4, 4]
print (list1)
print (list1.append ("python"))
print (list1)                    # [1, 2, 3, 4, 4, "python"]

→ list1 = [1, 2, 3]
print (list1.insert (1, 5)) # [1, 5, 2, 3]
print (list1)
print (list1.insert (4, 10)) # [1, 5, 2, 3, 10]
print (list1)

→ list1 = [1, 2]
print (list1.extend ([4, 5])) # [1, 2, 4, 5]
print (list1)
```



Ex:
```
list1 = [1,2,3]
list1.extend([10,20])
print(list1)  # [1,2,3,10,20]
list1.append(100)
print(list1)  # [1,2,3,10,20,100]
list1.insert(1,5)
print(list1)  # [1,5,2,3,10,20,100]
list1.insert(10,5) # [1,5,2,3,10,20,100,5]
print(list1) # out of range index value getting
               added at end of the list
```

append ex:
```
a = [1,2,3,[35,"a"],2.1]
a.append([24,"a"])
print(a)
print(len(a))
```
o/p: [1,2,3,[35,"a"],2.1,[24,"a"]]
6

→ list supports max and min

Ex:
```
list1 = [1,2,3]
print(max(list1)) # 3
print(min(list1)) # 1
```

→ count: It returns the number of times a specified element appears in the list. If it is not found it returns "0"

Ex:
```
list1 = [1,2,3,4,5]
print(list1.count(1)) # 1
```

→ reverse: It reverses the elements in list

Ex:
```
list1 = [1,2,3]
list1.reverse()
print(list1) # [3,2,1]
```

→ sort: It sorts the elements of a list in ascending order by default

Ex:
```
list1 = [10,1,20]
list1.sort()
print(list1) # [1,10,20]
```

If we want in descending order →
```
list1 = [10,1,20]
list1.sort(reverse = True)
print(list1)
      # [20,10,1]
```

→ remove: It removes the first occurence of specified value from the list. If value not found it raises a value error.

Ex:
```
list1 = [1,2,3]
list1.remove(1)
print(list1)  # [2,3]
```

→ POP: It removes and returns the element at the given index from a list. If no index is specified it removes and returns the last element.

Ex:
```
list1 = [1,2,3,4]
var1 = list1.pop()
print(var1)  # 4
print(list1) # [1,2,3]
```

→ count examples:

Ex:
```
a = [1,2,2,3,4,2]
print(a.count(2)) # 3
```

Ex:
```
fruits = ['apple','banana','apple','Grape','orange']
print(fruits.count('apple')) # 2
print(fruits.count('banana')) # 1
print(fruits.count('Grape')) # 1
```

Ex:
```
a = [1, True, False, 1, 0, True]
print(a.count(1))  # 4
print(a.count(True)) # 4
print(a.count(False)) # 2
```

Here True is consider as "1", false is consider as "0" and "1" is consider as "True", "0" is " " "false".

→ copy & clear:

Ex:
```
list1 = [1,2,3]
list2 = list1.copy()
print(list2)  # [1,2,3]
list1.clear()
print(list1) # [ ]
```

Tuple :

→ Tuple is a collection of elements in python which should be declared in parenthesis "( )"

→ Tuple is immutable data type once we defined the tuple we cannot modify it.

→ Tuple is ordered data type. It allows duplicate elements

→ Tuple is faster than list and it will occupy less memory space compared to list

→ we cannot define a single element in tuple, → Tuple take less memory

Ex: t1 = (1)
=> type(t1) # <class 'int'>

Ex: t2 = (1,) → we need to keep "," after element to define a tuple
=> type(t2) # <class 'tuple'>

→ Tuple supports indexing and slicing

Ex:
```
t1 = (1,2,3)
t2 = ("a","b","c")
t3 = (1) #int
t4 = ("a") # str
t5 = (1,) # tuple → to define tuple with single element
```

```
print(type(t1)) # <class 'tuple'>
print(type(t2)) #  "
print(type(t3)) # <class 'int'>
print(type(t4)) # <class 'str'>
print(type(t5))# <class 'tuple'>
```

```
print(t1[1]) #1
Print (t1[0:3]) # (1,2,3)
```

→ Supported methods (count, index)

```
t1 = (1,2,3,3)
print (t1.count(3)) # 2
print (t1.count(10)) # 0
print (t1.index(1)) # 0
print (t1.index(10)) # value error
```

→ it is used to get index of a given character.

→ To prove Tuple is faster than list below is the example

```
Ex: import sys
    t1 = (1,2,3)
    r1 = [1,2,3] → s1 = "a","b","c"
    print (sys.getsizeof(t1)) # 64
    print (sys.getsizeof(r1)) # 88
    print (sys.getsizeof(s1)) # 64
```

To define empty variables

```
→ str1 = ""
  str1 = str()

→ list1 = []
  list1 = list()

→ tuple1 = ()
  tuple1 = tuple()

→ set = {}
  set1 = set()
```

List → # del method →Ex:
```
a = [1,2,3,"a",[2,"a"]]
del a[4]
print(a) # [1,2,3,'a']
```

Ex:
```
list1 = [1,2,3]
del list1[0] # works based on index
print(list1) # [2,3]
```

```
#isinstance:
Syntax: isinstance(variable, type) # Bool >> True/false
print (isinstance(list1, tuple)) # False
print (isinstance (list1, list)) # True
print (isinstance ("abc", str)) # True
print (isinstance (1), tuple)) # False
print (isinstance ( (1,), tuple)) # True
print (isinstance ( {1, "a"}, set) # True.
```

Set

→ Set is a collection of elements in python and it is unordered data type

→ set won't support indexing, slicing

→ set won't allow duplicate elements

→ set is mutable datatype

→ set is defined in curly brackets {}

```
Ex: set1 = {1,2,3}
    print(set1) # {1,2,3}
    set2 = {1,2,3,1,2,3}
    print(set2) # {1,2,3}
```

#add method:

```
Ex: set1 = {1,2,4}
    set1.add(10) → It takes only one argument to add, if two given it shows Type error
    print(set1)  # {1,2,4,10}
```

#update method; It adds all elements from another set to current one.

```
Ex: set1 = {1,2,3}
    set2 = {4,5,6}
    set1.update(set2)
    print(set1)      # {1,2,3,4,5,6}
```

#clear method:

```
Ex: set1 = {1,"a",2.1}
    set1.clear()
    print(set1)   # set()
```

copy method

Ex:
```
set1 = {1, "a", 2.1}
set1.copy()
print(set1)  # {1, 2.1, "a"}
```

remove method

Ex:
```
set1 = {1, "a", 2.1}
set1.remove(1)
print(set1)  # {2.1, "a"}
```

→ If value is not there in set it will show key error.

pop method

Ex:
```
set1 = {"b", 4, "a", 2.1}
set1.pop()
print(set1)  # {"b", 4, "a"}
```

→ In pop if value is given it returns Type error. Because in set it takes no argument

discard method → used to remove an element

→ If element exist it removes it
→ " " does not exist it does nothing (no error)

Ex:
```
set1 = {1, "a", 2.1}
set1.discard(2)
print(set1)  # {1, "a", 2.1}
```

Ex:
```
set1 = {1, "a", 2.1}
set.discard("a")
print(set1)  # {1, 2.1}
```

more

* Set methods: union, intersection, difference, symmetric_difference

→ Ex: set1 = {1,2,3,4,5,10}
set2 = {3,4,5,6}

```
set3 = set1.union(set2)
print(set3)  # {1,2,3,4,5,6,10}
set4 = set1.intersection(set2)
print(set4)  # {3,4,5}
set5 = set1.difference(set2)
print(set5)  # {1,2,10}
set6 = set2.difference(set1)
print(set6)  # {6}
set7 = set1.symmetric_difference(set2)
print(set7)  # {1,2,6,10}
set8 = set2.symmetric_difference(set1)
print(set8)  # {1,2,6,10}
set1.update(set2)
print(set1)  # {1,2,3,4,5,6,10}
```

union: combine all elements from both sets - no duplicates. (set1 | set2 (or) set1.union(set2))

Intersection (set1 & set2 (or) set1.intersection(set2))
Returns only the common elements from both sets

Difference: (set1 - set2 (or) set.difference(set2))
→ Returns elements that are in the first set but not in the second.

symmetric_difference: (set1 ^ set2 (or) set1.symmetric_difference(set2))
→ Returns elements that are in either set, but not both

Type conversions: we can convert variable from one datatype to another

→ tuple → set, list
set → list, tuple
str → int, list ...
int → str
list → set, tuple

Ex: → int1 = 10
print (type(int1)) # int

→ str1 = str (int1)
print (str1) # 10
print (type (str1)) # <class 'str'>

→ str1 = "python"
print (int(str1)) # value error

→ str1 = "10"
print (type (int(str1))) # <class 'int'>
print (int(str1)) # 10

Ex: list1 = [1,2,3,4,5,1,2,3]

t1 = tuple(list1)
print (t1, type (t1)) # (1,2,3,4,5,1,2,3) <class 'tuple'>

s1 = set (list1)
print (s1, type (s1)) # {1,2,3,4,5} <class 'set'>

s2 = str(list1)
print (s2, type(s2)) # '[1,2,3,4,5,1,2,3]' <class 'str'>
print (s2 [0]) # [

str2 = "python"
print (list (str2)) # ['p','y','t','h','o','n']

Ex: list1 = ["a","b","c","a","v","b"]

```
set1 = set(list1)  #
print(type(set1)) # <class 'set'>
    print(set1) ------> # {'b','a','c','v'}
list1 = list(set1)  #
 print(list1) # ['a','b','c','v']
list2 = list(set(list1))  #
  print(list2) # ['a','c','b','v']
str1 = "abc"  #
print(list(str1)) # ['a','b','c']
    str1.split()
```

Dictionary

→ It is a set of key and value pair in python where keys are immutable and values are mutable.

→ When we want to map the data b/w two variables we can go for dictionaries

→ Dictionary allows duplicate values but we cannot have duplicate keys

→ Dictionary won't support indexing & slicing.

Syntax: dict1 = {key : value}

Ex: dict1 = {"a" : 20}

→ we can access values by its key.

```
print (dict1["a"]) # 20
print (dict1["b"]) # key error
```

Ex: dict1 = {"a":10, "a" : 20}

```
print(dict1) # {"a":20}
```

Ex: To add key-value pair #add

```
dict1 = {"a":10}
dict1["b"] = 20
print(dict1) # {"a":10, "b":20}
```

#delete

Ex: del dict1["a"]

```
print(dict1) # {"b":20}
```

Examples of type conversions:

String:

Integer to string.:

```
Ex: a=10
    b=str(a)
    print(b)  # 10
    print(type(b)) # <class 'str'>
```

Float to string:

```
Ex: a=10.1
    b=str(a)
    print(b)  # 10.1
    print(type(b)) # <class 'str'>
```

Boolean to string:

```
Ex: a=True
    b=str(a)
    Print(b)  # True
    print(type(b)) # <class 'str'>
```

list to string:

```
Ex: a=[1,2,3] (or) a=["a","b","py","",12.1,4]
    b=str(a)
    print(b) #[1,2,3] (or) ['a','b','py','',12.1,4]
    print(type(b)) # <class 'str'>
```

Set to string:

```
Ex: a={1,2,3}
    b=str(a)
    print(b) # {1,2,3}
    print(type(b)) <class 'str'>
```

tuple to String:

```
Ex: a = (1, 2, 3)
    b = str(a)
    print(b) # (1, 2, 3)
    print(type(b)) # <class 'str'>
```

Integer:

String to Integer:

```
Ex: x = "123"
    y = int(x)
    print(y)  # 123
    print(type(y)) # <class 'int'>
```

(or)

```
x = "23.7"
y = int(y)
print(y) # Error
```

float to Integer:

```
Ex: x = 21.7
    y = int(x)
    print(y) # 21
    print(type(y))#<class 'int'>
```

Boolean to Integer:

```
Ex: a = True
    b = int(a)
    print(b) # 1
    print(type(b)) # <class 'int'>
```

Errors : int("abc"), int([1,2]), int("23.7")

It will not convert list to int, set to int, tuple to int

float:

Integer to float:

```
Ex: a=10
    b=float(a)
    print(b) # 10.0
    print(type(b)) # <class 'float'>
```

String to float:

```
Ex: a="34.7"
    b= float(a)
    print(b) # 34.7
    print(type(b)) <class 'float'>
```

(or)

```
a="100"
b=float(a)
print(b) # 100
```

Boolean to float:

```
Ex: print(float(True))  # 1.0
    print(float(false)) # 0.0
```

List

String to List:

```
Ex: x="abc"
    print(list(x)) # ['a','b','c']
```

(or)

```
a="abc","dcb"
print(list(a))
o/p: ['abc','dcb']
```

Tuple to list:

```
Ex: x=(1,2,3)
    print(list(x)) # [1,2,3]
```

set to list:

```
Ex: x={1,2,3}
    print(list(x)) # [1,2,3]
```

Rangte to list:

```
x=range(5)
print(list(x)) # [0,1,2,3,4]
```

Dictionary to List

```
Ex: x = {"a":10, "b":20}
    print(list(x)) # ['a', 'b']
```

→ It doesn't convert int to list, float to list

→ Boolean obj is not iterable.

Tuple:

String to tuple:

```
Ex: a = "abc", "def"
    print(tuple(a)) # ('abc', 'def')
    print(type(tuple(a))) # <class 'tuple'>
```

Integer to tuple:

```
Ex: a = 10, 30
    print(tuple(a)) # (10, 30)
    print(type(tuple(a))) # <class 'tuple'>
```

Float to tuple:

```
Ex: a = 30.1, 456.10
    print(tuple(a)) # (30.1, 456.10)
    print(type(tuple(a))) # <class 'tuple'>
```

→ Boolean obj is not iterable

List to tuple:

```
Ex: a = ["a", 36.5, 34, "python"]
    print(tuple(a)) # ('a', 36.5, 34, "python")
```

Dict to tuple:

```
Ex: a = {'a':1, 'b':2}
    print(tuple(x)) # ('a', 'b')
```

→ Non-iterables : int, float, Boolean

Set

String to set:

Ex: a = "banana"
    print(set(a)) # {'a', 'b', 'n'} # not order

list to set:

Ex: x = [1, 2, 2, 3]
    print(set(x)) # {1, 2, 3}

Tuple to set:

Ex: x = (1, 2, 2, 3)
    print(set(x)) # {1, 2, 3}

Dictionary to set:

Ex: x = {'a': 1, 'b': 9}
    print(set(x)) # {'a', 'b'}

→ Non-iterables : int, float, Boolean

Differences of data types:

Feature	List	Tuple	set	Dict	int	Str
Description	A collection of elements that can be of any data type.	An ordered collection of elements like a list but unchangable	A collection of unique or unordered items	It is a set of key-value pair. Each key maps to a value.	whole numbers (+ve, -ve, or zero)	It is a sequence of characters
Mutability	Mutable you can change, add, remove elements after creation	Immutable cannot modify if once created	Mutable can add or remove elements	Keys-immutable values-mutable	Immutable cannot change the value once created	Immutable
Syntax	list=[1,2,3] defined with square brackets	tuple=(1,2,3) defined with paranthesis	set={1,2,3} defined with curly brackets	dict1={"a":1, "b":2} defined inside curly brackets	int1=0, int1=100	str1="python" defined within quotation marks
Allows duplicates	yes, same item can appear multiple times	yes	No (only unique element allowed)	Keys must be unique, values can allow duplicates	Not applicable	yes, same character can appear multiple times.
Indexing	Elements are accessed by index method Ex: list[0]	Supports indexed same as list	Doesnot support indexing	Access values using keys dict["name"]	Not needed - as it's a direct value	characters are accessed by index Ex: str[0]

order	follows ordered positions	follows order of items	unordered		NA	Maintains the order of characters, index based
Multiple data types	yes. can mix strings, num etc	yes, support mixed types	yes	yes - Keys & values can be different types	No	yes
used Methods	append(), extend(), insert(), POP(), sort(), remove()	count(), index()	add(), remove(), union(), interse-ction(), differenc() clear(), Symmet-ric_differen ce()	Keys(), values(), get(), update(), POP()		upper(), lower(), split(), replace(), count(), title(), None(), Swapcase()
use case	when you want to add/remove/ change elements freely	when you need a fixed ordered collection	when working with unique or unordered data	used for mappi-ng data	for calculations	used for formatting and text
Example	["apple", 2, True]	("x", 1, False)	{"apple", "Guava"} {1, 2, 3}	{"name": "Alice", "age": 25	10	"hello world"

Dictionary methods:

→ get, setdefault, update

get : Returns value if key exists, else None

Ex:
```
dict1 = {"a":20, "b":100}
print(dict1["a"])      # 20
print(dict1["c"])      # Keyerror
print(dict1.get("a")) # 20
print(dict1.get("c")) # None
print(dict1)
```

setdefault: Returns value if key exists, else it will add key with default value to the dictionary.

Syntax: dict1.setdefault(key, default)

→ default value in python is "None"

Ex:
```
print(dict1.setdefault("c", 1000)) # 1000
print(dict1)  # {"a":20, "b":100, "c":1000}
```

update: used to update one dictionary with another one.

Ex:
```
dict1 = {"a":10, "b":20}
dict2 = {"c":100}
dict1.update(dict2)
print(dict1)  # {"a":20, "b":20, "c":100}
```

→ keys, values, items

keys: To get all keys from dictionary

values: To get all values from dictionary

Ex:
```
dict1 = {"a":10, "b":100}
keys = dict1.keys()
values = dict1.values()
print(type(keys))   # <class 'dict_keys'>
print(type(values)) # <class 'dict_values'>
keys = list(keys)
values = list(values)
print(keys, values) # ['a','b'] [10,100]
```

items: To get both keys and values

Ex:
```
print(list(dict1.items())) #[('a',10),('b',100)]
```

→ copy, clear

copy: To create duplicate dictionary

Ex:
```
dict1 = {"a":10, "v":100}
dict2 = dict1.copy()
print(dict2) # {'a':10,'b':100}
```

clear:
```
dict1.clear()
print(dict1) # {}
```

↳ To remove all keys and values from dictionary

⇒ POP, Popitems

POP: removes key and value pair based on the key

→ Keyerror if key doesn't exists

POPitem: Bydefault Removes last key and value pair

Ex:
```
dict1 = {"a" : 10, "v" : 100}
dict1.pop("a")
print(dict1)   # {"v" : 100}
dict1.popitem()
print(dict1)   # {}
```

Another example of POPitem

Ex:
```
dict1 = {"a" : 10, "v" : 100}
dict1.popitem()
print(dict1)   # {'a' : 10}
```

Examples:

```
dict1 = {"a" : [1,2,3,4], "b" : "python"}
print(dict1["a"])   # [1,2,3,4]
print(dict1["b"])   # python
print(dict1["a"][0])  # 1
print(dict1["a"][-1])  # 4
```

→
```
dict1 = {"a" : [1,2,3,[10,20],4], "b" : "python"}
print(dict1["a"][3])   # [10,20]
print(dict1["a"][3][1])  # 20

#add key value pair c,(1,2,3,4)
dict1["c"] = (1,2,3,4)
print(dict1)  # {"a" : [1,2,3,[10,20],4], "b" : "python",
                "c" : (1,2,3,4)}
```

```
dict1["a"].append(5)
print(dict1)
    # {'a':[1,2,3,[10,20],4,5], "b":"python",
      "c":(1,2,3,4)}

dict1["a"][3].append(30)
print(dict1) # {"a":[1,2,3,[10,20,30],4,5], "b":"pyth
    -n", "c":(1,2,3,4)}

dict1["a"][3].insert(1,40)
print(dict1) # {"a":[1,2,3,[10,40,20,30],4,5],
    "b":"python",'c':(1,2,3,4)}


# to sort

dict1["a"][3].sort()
print(dict1) # {"a":[1,2,3,[10,20,30,40],4,5],
    "b":python", "c":(1,2,3,4)}


# to reverse
reverse_dict = dict1["b"][::-1]
Print (reverse_dict)  #nohtyp


Ex: dict1 = {"a":[1,2,3,[10,40,20,30],4,5], "b":"python",
=   "c":{"d":{"f":[1,2,3,4]}}}

print(dict1)
print(dict1["c"]) # {"d":{"f":[1,2,3,4]}}
print(dict1["c"]["d"] # {"f":[1,2,3,4]}
print(dict1["c"]["d"]["f"])  #[1,2,3,4]

#add 5 after 4
dict1["c"]["d"]["f"].append(5)
print(dict1["c"]["d"]["f"])  #[1,2,3,4,5]
```

to replace

Ex:
```
dict1 = {"a":10, "b":30}
dict1["c"]=40
dict1["b"]=50
print(dict1)  # {"a":10, "b":50, "c":40}
```

to add sublist [1,2,3]

Ex:
```
list1 = [10,20,30]
list1.append([1,2,3])

list1 = [10,20,30,[1,2,3]]
print(list1[0]) # 10
print(list1[-1]) # [1,2,3]
print(list1[-1][0]) # 1
list1[-1].insert(1,5)
print(list1) # [10,20,30,[1,5,2,3]]
list1[-1].pop(1)
print(list1) # [10,20,30,[1,2,3]]
list1[-1].reverse()
print(list1) # [10,20,30,[3,2,1]]
print(list1[-1][::-1])
                # [1,2,3]
```

Ex:
```
list1 = [[1,2,3],[4,5,6],[7,8,9]]
print(list1[0][1], list1[1][1], list1[2][1]) # 2 5 8
```

Ex:
```
list1 = [1,2,3]
print(sum(list1)) # 6
```

Ex:
```
list1 = [[1,2,3],[4,5,6],[7,8,9]]
list2 = []
sum_1 = sum(list1[0])
sum_2 = sum(list1[1])
sum_3 = sum(list1[2])
```

```
list2.append(sum_1)
list2.append(sum_2)
list2.append(sum_3)
print(sum(list2)) # 45
print(sum_1)  # 6
print(sum_2)  # 15
print(sum_3)  # 24
```

Ex:

=>

```
list1 = [1,2,3,(10,20,30),{"a":20,"b":30},{1,2,3}]

list1[4]["c"] = 40
print(list1) # [1,2,3,(10,20,30),{"a":20,"b":30,"c":40},
                                                {1,2,3}]

list1[4].popitem()
print(list1)# [1,2,3,(10,20,30),{"a":20,"b":30},{1,2,3}]

print(list1[-1]) # {1,2,3}
```

Ex:

=>

```
t1 = (1,2,3,[10,20,30])
print(t1[0])  # 1
Print(t1[-1]) # [10,20,30]

t1[-1].append(40)
print(t1)   # (1,2,3,[10,20,30,40])
```

Ex:

```
t1 = (1,2,3,{1,2,3})
t1[3].add(10)  #
print(t1)      # (1,2,3,{10,1,2,3})
```

Ex:

```
list1 = [1, 2, [7, 8, [4, 5, 6]]]
Print(len(list1))  #3
list1[2][2].insert(1,10)
list1[2].insert(2,10)
print(list1)
```

```
list1.insert(2,10)
Print(list1) # [1,2,10,[7,8,[4,5,6]]]
list1[3].insert(2,10)
print(list1)# [1,2,10,[7,8,10,[4,5,6]]]
list1[3][-1].append(10)
Print(list1) # [1,2,10,[7,8,10,[4,5,6,10]]]
```

Ex: dict1 = {"a": {"b": {"c": 30}}}

```
#add key value pair "d" 40
# add key value pair "c" 50
# modify the value of "c" 30 to 40
dict1["a"]["b"]["c"] = 40
print(dict1)
#add key value pair "d" 100 after "c" 40
# get all the keys of sub dictionary
```

```
# add key value pair "d" 40
dict1["d"] = 40
print(dict1) # {"a": {"b": {"c": 30}}, "d": 40}
# add key value pair "c" 50
dict1["c"] = 50
print(dict1) # {"a": {"b": {"c": 30}}, "d": 40, "c": 50}
# modify the value of "c" 30 to 40
dict1["a"]["b"]["c"] = 40
print(dict1) # {"a": {"b": {"c": 40}}, "d": 40, "c": 50
```

```
#add key value pair "d" 100 after "c":40
dict1["a"]["b"]["d"] = 100
print(dict1)
```

operators

→ They are special symbol or keyword used to perform operations on variables and values they are;

1. Arithematic operators
2. comparision operators
3. Logical operators
4. Assignment operators
5. Membership operators
6. Identity operators
7. Bitwise operators

① Arithematic operators: Used for basic mathematical operations.

operator	Description	Example
+	Addition	5+3=8
-	subtraction	5-3=2
*	Multiplication	5*3=15
/	Divison	5/2=2.5
//	Floor division	5//2=2
%	Modulus	5%2=1 (Remainder)
**	Exponential	2**3=8

Ex: x = 2
y = 4
z = x + y # 6
z = x - y # 2

```
z = x * y   # 8
z = y / x   # 2 [Actual value]
x = 5 // x  # 2 [Roundoff value]
x = y % x   # 0
z = x ** 3  # 8
```

(4) Assignment operator: used to assign values to variables

operator	-	Description	-	Example
=	→	Assign	→	x = 5
+=	→	Add and Assign	→	x += 3 (x = x + 3)
-=	→	subtract and Assign	→	x -= 2
*=	→	Multiply and Assign	→	x *= 4
/=	→	Divide and Assign	→	x /= 2

Ex:
```
x = 10
z = x + 10
```

```
x = x + 3
x += 3
```

(2) comparison operators: used to compare values

operator	-	Description	-	Example
==	→	Equal to	→	5 == 5 (True)
!=	→	Not Equal to	→	5 != 3 (True)
>	→	Greater than	→	5 > 3 (True)
<	→	less than	→	5 < 3 (false)
>=	→	Greater than or equal	→	5 >= 5 (True)
<=	→	less than or equal	→	3 <= 5 (True)

⑤ Membership operator : used to check elements in sequence

operator - Description - Example

in → present → 'a' in 'apple' → True

not in → Not present → 'x' not in 'apple' → True

Ex: x = "python"
=> "p" in x # True

```
listl = [1,2,3,4,5]
1 in listl # True
10 in listl # False
```

Ex: dictl = {"a":10, "b":20}
=>

```
"a" in dictl.keys() # True
20 in dictl.values() # True
30 in dictl.values() # False
30 not in dictl.values() # True
```

③ Logical operators : used to combine conditional statements

operator - Description - Example

and → True if both are true → True and false (false)

or → True if one is true → True or false (True)

not → Inverts the result → not true (false)

Ex: x = 10
=>

```
y = 20
x > 10 and y > 20   # False
x <= 10 and y == 20 # True
x == 10 and y != 20 # False
```

Ex: x = 10
y = 20

x % 2 = 0 or y % 2 = 0 # True
x > 10 or x % 2 != 0 # False
5 // 2 == 2 or y > 20 # True

(6) Identity operator: check if two variables point to the same object

operator	Description	Example
is	same object	x is y
is not	Not the same object	x is not y

To compare memory locations of two variables
To get memory location.

```
x = 10
id(x)
y = 20
id(y)
```

x is y :: True
x is not y :: False

How memory management works in Python?

In python memory management will be handled automatically by python memory manager.
→ whenever we create variable or object python will automatically allocates memory for it.

→ python uses heap space to store all variables and objects. (python interpreter will manage it)

→ Ex: x = 10
y = 10

python will store actual value in its memory and it will add references to it.

→ whenever we change the value of a variable x=20, y=1 it will mark previous entry as stale (no longer available) entry and all these entries will be automatically deleted in the background. This process is called as Garbage collection (gc)

→ python will handle garbage collection automatically

→ we can also explicitly perform garbage collection by importing a module called gc

⟹ import gc
gc.collection()

⑦ Bitwise operators: operate on binary numbers

operator	- Description	- Example
&	→ AND	→ 5 & 3 = 1
^	→ XOR	→ 5 ^ 3 = 6
~	→ NOT	→ ~5 = -6
<<	→ Left shift	→ 5 << 1 = 10
>>	→ Right shift	→ 5 >> 1 = 2

String formatting:

```
x=10
y=20
```

→ we can format the strings in different ways

1. using .format() # < python 3.0
2. using f"string" # >= python 3.0

```
print(f"the value of x is {x} and y is {y}")
print("The value of x is", x)
print(f"the sum of {x} and {y} is {x+y}")

str1=f"the value of x is {x}"
print(str1)

str1="x value is {} and y value is {}".format(x,y)
print(str1)
```

String concatenation:

```
str1 = "python"
str2 = "3"
str3 = str1 + str2
print(str3)
```

→ for integer it will add

```
Ex: x=10
    y=20
    z= x+y
    print(z) # 30
```

Ex:
```
list1 = [1,2,3,4]
list2 = [4,5,6]
print(list1 + list2) # [1,2,3,4,4,5,6]
```

Ex:
```
set1 = {1,2,3}
set2 = {4,5,6}
print(set1 + set2)  # TypeError
```

Ex:
```
t1 = (1,2,3)
t2 = (4,5,6)
print(t1 + t2)  # (1,2,3,4,5,6)
```

Ex:
```
dict1 = {"a":10}
dict2 = {"b":20}
print(dict1 + dict2) # Type Error
```

conditional statements:

→ In python conditions are used to check something and make decisions

→ we can use if, elif and else block the

Syntax:
```
if <condition>:
    <BLOCK>
else:
    <BLOCK>
```

→ if the condition returns True the code inside if will get executed.

→ if it returns false the code inside else will get executed

Ex: X = 10
=> Y = 20
```
if x > y :
    print (True)

else:
    print(false)
```
o/p: False

→ we can check multiple conditions by adding more elif blocks

Ex: X = 100
=>
```
if x > 100:
    Print(" ")
elif  x > 50:
    Print(" ")
elif x = = 10:
    Print(" ")
else:
    print(" ")
```
→ It will be the o/p

→ even & odd

Ex: X = 20
=>
```
if  x % 2 = = 0:
    print("even")

else:
    print("odd")
```
o/p: even
=>

→ largest number

Ex:
```
x=10
y=20
z=30
if x>y and x>z:
    print(x)
if y>x and y>z:
    print(y)
else:
    print(z)
```
o/p: 30

Ex:
```
str1="python" # even, last element in n
if len(str1)%2==0 and str1[-1]=="n":
    print(True)
```

To check given condition is string

```
str1="python"
if type(str1)=="str":
    print("It is str datatype")
elif type(str1)=="list":
    print("It is list datatype")
else:
    print("None")
```
o/p: It is str datatype

input():

used to read inputs from user in runtime

Syntax: x = input()

x = input ("Enter some number")

→ input() function will return data in string format.

Nested :

Ex:

```
username = input ("username :: ")
password = input ("password:: ")
if username == "root":
    if password == "winteck@123":
        Print("username and password are valid")
    else:
        print ("invalid password")
else:
    username = input ("incorrect username enter valid
                       username")
    if username != "root":
        print("still wrong")
```

take input from user in int format

check if it is positive number or not

check if it is even number or not

chek number is in btw 10 to 100

```
a = int(input())
if a<0:
    print("positive")
else:
    print("negative")
```

X WRONG

```
if user>0:
    print("positive")
    if user%2==0:
        print("positive")
        if a>10 and a<100:
            print("valid")
        else:
            print("invalid")
    else:
        print("invalid")
else:
    print("negative")
```

~~if a>10 and a<100:~~

(A)
```
user = int(input())
if user>0:
    print("positive number")
    ~~if user>10 and user<100:~~
        ~~print~~
    if user%2==0:
        print("even number")
        if user>10 and user<100:
            print("it is in b/w 10 and 100")
        else:
            print("it is not in b/w 10 and 100")
    else:
        print("odd number")
else:
    print("negative number")
```

o/p: 30
positive number
even number
it is in b/w 10 and 100

-90
negative number

57
positive number
odd number

Loops

for loop:

Ex: list1 = [1, 2, 3, 4, 5, 6]

Syntax: for iterable in sequence:
    <Block_of_code>

```
for i in list1:
    print(i)  # 1 2 3 4 5 6
```

→ loops are used to iterate over the given sequence

→ There are two loops in python
  ① for loop
  ② while loop

① for loop: for loop will iterate over the elements in sequence
syntax and example is

Ex: str1 = "python"

```
for i in str1:
    print(i)
```

list1 = ["python", 1, 2, 4, {1, 2, 3}]

```
for i in list1:
    print(f"value: {i}, type: {type(i)}")
```

```
list1 = ["python", 1, 2, 4, {1,2,3}]
for i in list1:
    if type(i) == int:
        print(f"Integer is {i}")
```

```
# if element is string print len of element
list1 = ["python", 1, 2, 4, {1,2,3}]
for i in list1:
    if type(i) == str:
        print(f"value: {i}, length: {len(i)}")
```

```
# literate the over the sequence and print type of
each element
# print if only element is int
# if the element is string print len of element
```

Ex:
```
list1 = [1, 2, 3, "python", {1,2,3}, -4]
for i in list1:
    print(f"the value is {i}, and type is {type(i)}")
    if type(i) == int:
        print(f"the integer is {i}")
        if type(i) == str:
            print(f"the length is {len(i)}")
        else:
            print("not a valid data")
    else:
        print("no valid data")
```

O/P:
```
the value is 1, and type is <class 'int'>
the integer is 1
not a valid data
the value is 2, and type is <class 'int'>
the integer is 2
```

not a valid data
the value is 3 , and type is <class 'int'>
the integer is 3
not a valid data
the value is python and type is <class 'str'>
no valid data
the value is {1,2,3}, and type is <class 'set'>
no valid data
the value is -4 and type is <class 'int'>
the integer is -4
not a valid data

Ex:

```
dict1={"a":10,"b":20,"c":30}
list_keys=[]   #a b c
list_values=[] #10 20 30
for key in dict1.keys():
        list_keys.append(key)
for value in dict1.values():
        list_values.append(value)

print(list_keys)
print(list_values)
```

o/p: ['a','b','c']
[10,20,30]

Range function

→ Range is built in function in python which is used to generate sequence

Syntax: range (start, end)

Ex: list1 = list (range (1, 100))
print (list1) # It will print 1 to 100 numbers

SKIP

range (start, end, step)

Ex:-2: list2 = list (range (1, 10))
print (list2) # [1, 2, 3, 4, 5, 6, 7, 8, 9]
list2 = list (range (1, 10, 2))
print (list2) # [1, 3, 5, 7, 9]

Ex:-3: for i in range (1, 10):
print (i) # → 1 2 3 4 5 6 7 8 9
for i in range (10):
print (i) # → 0 1 2 3 4 5 6 7 8 9
for i in range (1, 10, 2):
print (i) # → 1 3 5 7 9
for i in range (0, 10, 2):
print (i) # → 0 2 4 6 8
for i in range (1, 10, 4):
print (i) # → 1 5 9

Ex:

```
Str1="Python version 3.0@"
list_char=[ ]
list_space=[ ]
list_special_char=[ ]
list_digit=[ ]
for i in Str1:
    if i.isalpha():
        list_char.append(i)
    elif i.isspace():
        list_~~char.a~~space.append(i)
    elif i.isdigit():
        list_digit.append(i)
    else:
        list_special_char.append(i)
print(list_char)
print(list_space)
print(list_digit)
print(list_special_char)
```

o/p: ['P','y','t','h','o','n','v','e','r','s','i','o','n']
['',' ']
['3','@']
['.','@']

Ex:

```
Str1="aaabbccccccdd"
b=[ ]
for i in Str1:
    if Str1.count(i)>1 and i not in b:
        b.append(i)
        print(i)
```

o/p: a
b
c
d

Ex:

Loop control statements

~~break: It will bre~~

→ Loop control statements are used to control the flow of execution inside the loop

they are → break → continue → Pass

→ Break: break will terminate the loop permanently.

Ex:
```
list1 = [1,2,3,4,5]
for i in list1:
    if i==3:
        print(i)
        break
print("Remaining block")
```

O/P:
```
1
2
Remaining block
```

→ continue: continue will skip current iteration and executes the next iteration

Ex:
```
list1 = [1,2,3,4,5]
for i in list1:
    if i==3:
        continue
    else:
        print(i)
Print("Remaining block")
```

O/P:
```
1
2
4
5
Remaining block
```

→ Pass: pass will not make any impact on code. It is used to define empty loops functions and classes.

It will avoid python to trash for syntax errors when we define empty block of code

Ex: for i in range(10):
        Pass
    while True:
        pass
    class cls:
        pass
    def fun1():
        pass

Ex: list1 = [10, 20, 30, 11, 13, 200]
=
for i in ~~range(0,14)~~ list1:
    if i%2 != 0:
        break
    print (i)

---

for i in ~~range(0,10)~~ list1:
    if i % 2 != 0:
        continue
    print(i)

Ex: list1 = [1, 2, 3, 4, 10, 15, 18, 19]
=
count_even = 0
count_odd = 0
for i in list1:
    if i%2 == 0:
        count_even = count_even+1

    else:

        count_odd = count_odd + 1

print (count_even)
print (count_odd)

~~a=int(input())~~ first 5 even numbers

~~b=0~~

Ex:
```
for i in range(0,100):
    if i%2==0:
        print(i)
        ~~break~~
        count+=1
        if count==5:
            break
```

Print prime numbers in the range of 50
Print largest number from list
Print second largest number from list

largest number from list:

```
list1=[10,20,177,16]
max1=10
for i in list1:
    if i>max1:
        max1=i

Print(max1)
```

o/p: 177

Second largest number

```
list1=[10,20,177,16]
max1=0
max2=0
for i in list1:
    if i>max1:
        max2=max1
        max1=i
    elif i>max2:
        max2=i

Print(max2)
Print(max1)
```

o/p: 20
177

To iterate over a indexes of a sequence

```
list1 = [10,20,177,16]
for i in range(len(list1)):
        Print(list1[i])  # 10 20 177 16
        Print(i)         # 0 1 2 3
```

all the elements in list1 are there in list2 or not

Ex:
```
list1 = [10,20,30]
list2 = [20,30,40]
for i in list1:
    if i not in list1:
        print("Mismatch")
        break
```

find diff b/w two list (HM)

Nested range

```
for i in range(5):
    Print(i)
    for j in range(5,10):
        print(i,j)
```

Ex:
```
for i in range(5):
    print(i)
    for j in range(5,10):
        print(j)
        if j==5:
            break
```

Enumerate:

→ Enumerate is a built-in function in python which is used to access both index and actual element of a sequence.

→ output of enumerate function will be list of tuples

Ex: list1 = [10, 20, 30, 40, 50, 60]
print(list(enumerate(list1)))

[(0,10), (1,20), (2,30), (3,40), (4,50), (5,60)]

By using iterate

Ex: list1 = [10, 20, 30, 40, 50, 60]
for i,j in enumerate(list1):
    print(i,j)

o/p:
0 10
1 20
2 30
3 40
4 50
5 60

enumerate by using dict method

Ex: list1 = [10, 20, 30, 40, 50]
print(dict(enumerate(list1))) o/p:

using for loop

Ex: list1 = [10, 20, 30, 40, 50]
dict1 = { }
for i,j in enumerate(list1):
    dict1[i] = j

print(dict1) o/p: {0:10, 1:20, 2:30, 3:40, 4:50, 5:60}

same

```
# swap inside list by using index
list1=[10,20,30,40,50]
list1[1],list1[3]=list1[3],list1[1]
print(list1)  # [10,40,30,20,50]

# difference b/w two lists
list1=[1,2,3,4,5]
list2=[4,5,6,7]
diff1=[]
for item in list1:
    if item not in list2:
        diff1.append(item)

print(diff1)
```

1 1 is not in list2 True
2 True
3 True
4 False
5 False

o/p: 1,2,3

Bubble sort:

```
Ex: list1=[77,25,1,99,10]
    n=len(list1)
    for i in range(n):
        for j in range(0,n-i-1):
            if list1[j]>list1[j+1]
                list1[j],list1[j+1]=list1[j+1],
                                    list1[j]

    print(list1)   # [1,10,25,77,99]
```

Zip(): If we want to combine two lists and generate list

```
Ex: list1=[1,2,3]
    list2=[4,5,6]
    print(list(zip(list1,list2)))
      # [(1,4),(2,5),(3,6)]
```

To create dict from two list

Ex:
```
dict1 = dict(zip(list1, list2))
print(dict1)
```

```
dict2 = {}
for i,j in zip(list1, list2):
        dict2[i] = j
```

Ex:
```
list1 = [10,20,30,60]
list2 = [30,40,50,80]
list3 = []
for item in list1:
    if item in list2 and not in list3:
        list3.append(item)
print(list3)
```

wrong

Ex:
```
list1 = [10,20,30,60]
list2 = [30,40,50,80]
list3 = []
for i,j in zip(list1, list2):
    if i not in list2:
        list3.append(i)
    if j not in list1:
        list3.append(j)
```

Ex: `set(list1).symmetric_difference(set(list2))`

Packing & unpacking

```
list1 = ["a","b","c"]
list2 = [1,2,3]
list3 = zip(list1, list2)
list3 = [
```

unpack

```
list3 = [(1,4),(2,5),(3,6)]
list1, list2 = zip(* list3)
print(list1)
print(list2)
```

Ex:
```
list1 = [10, 10, 50, 60, 20, 50]
list2 = [ ]
for i in list1:
    if i not in list2:
        list2.append(i)

print(list2)
```

Ex:
```
for i in list1:
    if list1.count>1:
        i.remove
```

Ex:
```
list1 = [10, 10, 50, 60, 20, 50]
for i,j in enumerate(list1):
    if list1.count(j)>1:
        list1.pop(i)
print(list1) # [10, 50, 60, 20]
```

Ex: Str1 = "abc"
Str2 = "123"
Str3 = ""
for i,j in zip(Str1, Str2):
    #breakpoint() # pdb.settrace()
    temp = i+j
    Str3 = Str3 + temp
Print(Str3) # a1b2c3

while :

→ As long as given condition is true, the block of code in while loop will be executed.

Ex: while <condition>:
        <Block>

→ while loop will be used to run infinite loops.

Ex: a=1
    while a<5:
        print("while Block")
        a=a+1

#run infinite loops
while True:
    print("Hello") # infinite loops.

Ex: Str1 = "abc"
Str2 = "123"
Str3 = ""
for i,j in zip(Str1, Str2):
    #breakpoint() # pdb.settrace()
    temp = i+j
    Str3 = Str3 + temp
Print(Str3) # a1b2c3

while :

→ As long as given condition is true, the block of code in while loop will be executed.

Ex: while <condition>:
        <Block>

→ while loop will be used to run infinite loops.

Ex: a=1
while a<5:
    print("while Block")
    a=a+1

#run infinite loops
while True:
    print("Hello") # infinite loops.

Comprehensions:

used to optimize the code and reduce no. of lines by writing block of code in a single line

Ex:: 1. List 2. Dict 3. Set

① List comprehension:

the output of list comprehension is in list format.

Syntax: list_out = [output/ <loop> <condition>]
expression

without using list comprehension:

```
list2 = [ ]
for i in range(10):
    if i%2!=0:
        list2.append(i)
```

with using list comprehension:

```
list2 = [i for i in range (10) if i%2 != 0]
print (list2)
```

Another ex:
```
list1 = ["abcd"]
for i in list1:
    for j in i:
        print (j)
        list2.append(j)
```

→
```
list2 = [j for i in list1 for j in i]
print (list2)
```

→
```
list1 = [i for i in range (10)]
print (list1)
```

→ list1=[1,2,3,4] ✗
b=[]
for i in list1:
    b.append(list1

→ list1=[1,2,3,4]
list2=[i**2 for i in list1]
print(list2)

→ list1=[[1,2],[2,3],[3,4]]
list2=[j for i in list1 for j in i]
print(list2)

→ list1=[[1,2,3],[3,4,1],[5,6,7]] (H.W)
remove duplicates ↑ unique numbers in new list

② Dictionary comprehension:

the output of dictionary comprehension is in dictionary format

Syntax: dict1={key:value <loop> <condition>}

Ex: list1=["a","b","c","d"]
list2=[1,2,3,4]
dict2={i:j for i,j in zip(list1,list2)}
print(dict2)

→ ~~dict3={"a":4}~~
dict3={i:j**2 for i,j in zip(list1,list2)}
print(dict3)

→ dict1={"a":10,"b":20,"c":88,"d":15}
b=[]
for i in dict1.items():
    if i>15:
        b.append(i)
print(b)

~~→ Swap Keys and Values~~

Ex: list1 = [[1,2,3], [3,4,1], [5,6,7]]

With out comprehension

```
b = set()
for i in list1:
    for j in i:
        b.add(j)
Print(list(b))        # [1,2,3,4,5,6,7]
```

With comprehension

```
list2 = list(set(j for i in list1 for j in i))
print(list2)          # [1,2,3,4,5,6,7]
```

Swap Keys and values

```
dict1 = {"a":10, "b":20, "c":88, "d":15}
dict2 = {}
for i,j in dict1.items():
    i,j = j,i
    dict2[i] = j
print(dict2)
```

With comprehension (swap)

```
dict1 = {j:i for i,j in dict1.items()}
print(dict1)
```

With comprehensions (unique numbers)

```
list1 = [[1,2,3], [3,4,1], [5,6,7]]
b = [j for i in list1 for j in i]
c = [i for i in b if b.count(i)==1 and i not in c]
print(c)
```

function

→ functions are ~~block~~ reusable block of code in python which takes some arguments and apply the algorithm and returns the output

→ functions are mainly for code reusability

→ once we define a function we can reuse it within the same python file (or) we can import it in another python script also

→ functions are defined with def keyword.

→ function arguments are not mandatory.

syntax: def <function_name>:
    <block_of_code>

Define a function:

```
def func1():
    print("Hello world")
func1() # calling function    # Hello world
```

function with arguments

```
def add(x,y):
    print(x+y)

add(10,20)          # 30
add(100,200)        # 300
add(-100,-200)      # -300
```

return statement

```
def add(a,b,c):
    return a+b+c

c=add(10,20,30)     # 60
Print(c)
```

→ return used to store the function output and send it back to the column

→ function arguments are (inputs to the function) during function execution

```
→ def funcl(x,y):
        if x>y:
            return x
        else:
            return y
        print("Hello")

output = funl(10,20)          # 20
Print(output)
```

→ Return will be called only once for function once return executes remaining block of code will be no run, stops the function

check given number is even number or not

```
→ def funcl(x):
        if x%2==0:
            return "even"
        else:
            return "odd"

output = funcl(10)            #even
print(output)
```

```
→ def check_datatype(x):
        return type(x)

c = check_datatype(10)
d = check_datatype("funcl")
e = check_datatype([1,2,3])
print(c)
print(d)
Print(e)

# <class 'int'>
  <class 'str'>
  <class 'list'>
```

check string contains digit

→ str1="s#t100"
if not str1.isalpha():
    print(str1) # s#t100

→ def check_digit_instr(str1):
    if not str1.isalpha():
        print(str1)
        #return True # True str contains numbers
    else:
        #return False
        print("No digit") # str1 None str1

print(check_digit_instr("str1"))
if check_digit_instr("str1"):
    print("str contains numbers")

difference b/w print & return ex: when we need to
to print something/ use print & return.

#8<msg>#8
len(msg)=# <msg> len(msg)

hello
#####hello#####
#a#
##ab##

#
*
* *
* * *
* * * *

#
1
12
123
1234

#
*
* *
* * *
* * * *

Difference between print vs return

print	return
→ The purpose of print is to display output to the screen	→ The purpose of return is to sends back a value from a function to the caller
→ It is used for debugging, displaying results	→ It is used for function logic, storing and further processing
→ It is used anywhere in the code	→ It is used inside a function
→ It returns None	→ It returns a specific value [int, string, list etc]
→ It does not change or pass values	→ It passes values back, and can end function execution.

when to use print:

use "print" when:

→ you want to display output to the user
→ you are debugging your code
→ you don't need to reuse the value later

when to use return:

use "return" when:

→ you're writing a function
→ you want to use the result later
(store it, pass it to another function).
→ you want your function to act like a calculation or decision-maker

#

```
str1=input()
b=len(str1)
print("#"*b+str1+"#"*b)
```

i/p: dhanuja

o/p: #######dhanuja#######

i/p: a

o/p: #a#

```
n=int(input())
for i in range(n):
    for j in range(i+1):
        print("*",end=" ")
    print()
```

i/p: 4

o/p:

```
*
* *
* * *
* * * *
```

```
n=int(input())
for i in range(n):
    for j in range(n-i-1):
        print(" ",end="")
    for j in range(i+1):
        print("*",end=" ")
    print()
```

i/p: 4

o/p:

```
   *
  * *
 * * *
* * * *
```

```
n=int(input())
for i in range(n):
    p=1
    for j in range(i+1):
        print(p,end=" ")
        p+=1
    print()
```

i/p: 5

o/p:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

① Positional Arguments:

→ Positional Arguments are mandatory in python
→ we need to pass the arguments in same position as defined in function def

```
Ex: def func1(x, y):
        print(f"The value of x is {x} and y is {y}")
        return x+y
    func1_out = func1(1000, 2000)
    func2_out = func1(100, 200) # The value of x is
                                  100 and y is 200

    x=100
    y=200
    func3_out = func1(y, x) # The value of x is 200
                              and y is 100
```

② Keyword Arguments:

→ Keyword Arguments are mandatory in python
→ while defining a function, arguments remains as position
→ while calling function we can make argument as keyword based

```
Ex: def func1(x,y):
        print(f"The value of x is {x} and y is {y}")
        return x+y
    out = func1(y=300, x=100)
    print(out)  # The value of x is 100 and y is
                  300
                  #400
    print(func1(100, x=200))  #error
    print(func1(x=100,200))  #error
          print(func1(100,y=200))  # correct
```

```
def func1(x,y):
    print(f"the value of x is {x} and value of
                                y is {y}")

    return x+y

out=func1(100,y=200)
print(out) # the value of x is 100 and value of y is 200
#300
def func1(x,y):
    """
      args :: pos x,y (int)   } -> docstring
      return int
    """
    print(f"the value of x is {x} and value
                        of y is {y}")

    return x+y
#invalid function call
out=func1(300,x=100) #TypeError: func1() got
                      multiple values for argument 'y'

def func1(x,y):
    print(f"the value of x is {x} and y is {y}")
    return x+y
#invalid function call
out=func1(x=100, 200) # syntaxerror : positional
                        argument follows keyword argument
```

→ # Always non positional arguments must be followed by positional arguments

③ Default Arguments:

→ Default arguments are not mandatory in Python.

→ def func1(x=100, y=200):
    return x+y
print(func1()) # 300
Print(func1(x=300, y=1000)) # 1300
print(func1(1000, 2000)) # 3000

→ To make function arguments as optional we can overwrite the values during function call.

→ def func1(b, x=100, y=200):
    return x+y

→ def func1(x=100, b, y=200):
    return x+y # syntax error: non-default argument follows default argument

(4) variable length positional arguments [*args]:

→ these are optional
→ To pass N no. of positional arguments
→ the o/p of var len pos arguments is tuple

Ex: def func1(x,y):
    return x+y
func1(1,2,3,4) # fun1() takes 2 positional arguments but 4 were given.

→ def func1(*args):
    return args
out=func1(1,2,3,4)
print(out) # (1,2,3,4)
out = func1()
print(out) # ()

```
→ def func1 (x, y, z=100, *args):
      return args   # (1,2,3)
    # return x+y+z  # (200,30,40)
      print (x, y, z, args) # 10 20 200 (1,2,3)
print (func1 (10,20, 200,1,2,3))

→ def func1 (*args, x=100):
      return args, x  # (10,20,30,200)
print (func1 (10,20,30, x=200))
```

⑤ variable length keyword Arguments [**args]:

→ these are not mandatory
→ To pass N no. of keyword arguments
→ the output of var len pos arguments will be in dictionary

```
Ex: def func1 (**kwargs):
        return kwargs
        return kwargs.keys()
      # return kwargs.values()
print (func1 (a=10, b=20))  # {'a':10, 'b':20}
print (func1 ())            # {}
```

{'a':100, 'b':200} → dict_keys(['a','b'])
{} → dict_keys([])

dict_values([100,200])
dict_values([])

Scope of variables:

→ order must be positional, default, *args, **kwargs

① Global variable
② Local variable

① **Global variable**: the scope of global variable will be across the functions/python script

② **Local variable**: the scope of local variable will be limited to that particular function

Ex: x=10 # Global variable

```
def func1():
    x=20 # local variable
    return x
print(func1()) # 20
print(x) # 10
```

→ we can change the scope of variable to global by using "global" keyword

Ex:

```
def fun1():
    global x
    x=20
    return x
                # 20
fun1()
print(x)
```

Ex:

```
x=100
print(x)
def fun1():
    global x
    x=20
    return x

fun1()              # 100
print(x)            # 20
```

Module: Module is a reusable block of code in python

→ Group of functions/classes in a single py file is called as module

→ Any python file with functions can be

considered as a module

Ex:
```
def sample_fun1():
    return None
def sample_fun2():
    return None
def sample_fun3():
    return None
def sample_fun4():
    return None
```

Why use module?
→ Reusability → write once use anywhere
→ organization → keep code in logical parts
→ Maintainability
↓
smaller, cleaner files

module1.py :: Module name is module1

test1.py

```
test1
    fun1()
    fun2()
    ..
    ::
    funn()
```

<All the above functions belongs to test1 module>

~~Mod~~

import <module_name>

→ we can import module by its name
→ once we import a module all the functions/ classes/variables will be available locally with the python script

Syntax:: # import <module_name>

```
import test1
import test2

test1.fun1()
#to call function
```

we can call function with module.fun_name

```
#print(test1.fun1())
# test1.fun2()
```

Ex: args1.py → filename

```
def func1(a,b):
    return a+b
func1(10,20)
```

args2.py → filename

```
def func1(a,b):
    return a-b
func1(100,20)
```

sample.py → filename

```
import args1
import args2
print(args1.func1(10,20))   # 30
print(args2.func1(100,80))  # 80
```

write a function that adds two numbers and returns the result

```
→ def func1(a,b):
      return a+b
  c=func1(10,20)
  print(c)
```

o/p: 30

write a function that checks if a number is even or odd

```
→ def func1(a):
      if a%2==0:
          return "even"
      else:
          return "odd"
  c=func1(1) #10
  print(c)
```

o/p: odd
#even

write a function that takes a list of numbers and returns the sum

→
```
def func1(a):
    sum=0
    for i in a:
        sum=sum+i
    return sum
c=func1([1,2,3,4])
print(c)
```
o/p: 10

write a function to calculate the factorial of a number

→
```
def func1(a):
    fact=1
    for i in range(1,a+1):
        fact=fact*i
    return fact
c=func1(5)
print(c)
```
o/p: 120

write a function that returns the maximum of three numbers

→
```
def func1(a):
    max1=0
    for i in a:
        if i>max1:
            max1=i
    return max1
c=func1([1,2,3])
print(c)
```
o/p: 3

→
```
def func1(a,b,c):
    return max(a,b,c)
c=func1(1,2,3)
print(c)
```
o/p: 3

< Description >
Function name
owner
Date of creation
actual work

@winteck 24/7/2025 Adding fun1 function to module1 fun
@winteck 26/7/2025 Fixed bug in fun1 fun
@empll 28/8/2025 Add new function for chrome chrom
functionality e_fun

Types of Modules:

1. user defined module:
   Ex: test1.py
2. python Inbuilt module:
   Ex: os, sys, time, numpy, pandas
3. user installed module:
   Ex: pytest, paramiko

PYPI :: To install modules
"PiP" :: python package installer
Syntax :: pip install package_name
Ex :: pip install pytest

import pytest

Install - pip install pytest
list - pip list
uninstall - pip uninstall pytest
upgrade - pip upgrade pytest

→ module1
    test1
    test2
    test3
    te
    create_raid_file_system_mount_check_output

import module1

>> To import particular function from a module
from module1 import test2

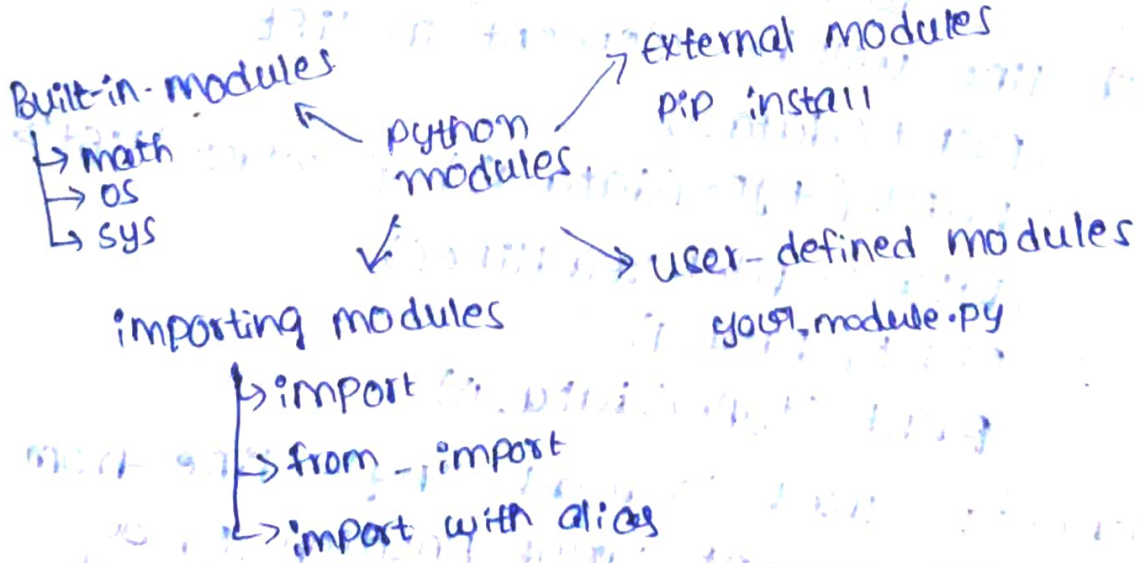
>> To import multiple functions from a module
from module1 import test2, test1

>> To import all functions from a module
from module1 import *

>> To create shortcut function alias
from module1 import test1 as t1
from module1 import create_raid_file_system_
    mount_check_output as raid_OPS

→ To print all functions in a module

```
import module3
print(dir(module3))
```



File Handling: used to handle files in python.

syntax to open a file:

```
open(filename, access_mode)
```

Access modes:

- w :: write → Ex: open(filename, "w")
- r :: read → Ex: open(filename, "r")
- a :: append → Ex: open(filename, "a")

File Handler: used to handle the files

we can control the files by using file handler

```
file_handler = open(filename, file_mode)
fl = open(file1, "r") # to open a file in read mode
```

Ex::

```
file1 = open("file1.txt", "r")
print(file1)
data = file1.read()
```

read():: used to read all file contents to a string format

```
print(data)
print(type(data))
data_1 = file1.readlines()
```

readlines():: used to read string contents to a list format,

→ each line will be element in list

```
print(data_1)
print(type(data_1)) # <class 'list'>
data_1 = file1.readline()
print(data_1)
print(type(data_1))
```

readline():: used to read single line from the current cursor position to a string format

```
file1 = open ("file1.txt", "w")
print (file1.read()) # io.UnsupportedOperation: not readable

file1.write ("This is first line\n")
file1.write ("This is second line\n")
lines = ["a\n", "b\n", "c\n"]
file1.writelines (lines)
file1.close()

#function to read data and return file contents
file1 = open ("file1.txt", "r")
data = file1.read()
def display_file_contents (file_name):
    f1 = open (file_name, "r")
    return f1.read()
display_file_contents (file1)

⇒ def display_readfile (file_name, mode = "read"):
    f1 = open (file_name, "r")
    if mode == "readlines":
        return f1.readlines()
    elif mode == "readline":
        return f1.readline()
    return f1.read()

def search_keyword (filename, keyword):
    file_content = display_file (file1)
    if keyword in file_content:
        return True

    else:
        return False

def count_lines_file (file_name):
    contents = display_file (file_name, mode = "readlines")
    return len (contents)
```

```
def get_first_line(filename):
    line=display-file (file_name, mode="readline")
    return line

def no_of_characters (filename):
```

seek() :: To move the cursor position
tell() :: To get the cursor position

```
file1 = open ("file1.txt", "r")
print (file1.tell())
file1.seek(5)
print (file1.tell())
print (file1.read())   # after 5th character
```

Content Manager:

→ content Manager will automatically manage the resource.

Ex :: with open(), we need not to close the file explicitly [like with f1.close()]

```
Ex ::
with open ("file2.txt", "r") as file_handler:
    data = file_handler.read()
    lines = file_handler.readlines()
    line = file_handler.readline()

print(data)
with open ("file2.txt", "a") as f1:
    f1.write(data)
```

```
# copy first 4 lines to a file
# function to return lines with only
  lowercase characters.
# function to return no. of lines with >
                              10 characters
```

#copy first 4 lines to a file

```
with open ("file2.txt", "r") as f1:
    lines = f1.readlines()
with open ("file2.txt", "a") as f1:
    f1.writelines(lines[:4])
```

JSON: Javascript object Notation

→ we use JSON files to read test inputs

→ we can load json files with python inbuilt "json" module

→ json.load() :: To load json data as dictionary data.

```
import json
with open ("file2.json","r") as json_data:
    data = json.load(json_data)

print(type(data))
print(data)
```

file2.json

```
{"key" : "value",
    "key1" : "value1",
        "key2" : "value2", "key3" : [1,2,3,4] }
```

json.loads() :: To load dictionary data from string

```
Ex :: json_string = '{"a":10", "b": 25, "c": "30"}'
      data = json.loads(json_string)
```

function to return no. of lines with >= 10 characters

```
def func1():
    count=0
    with open("details.txt","r") as f1:
        for i in f1:
            if len(i)>10:
                cont=count+1

    return count

c=func1()
print(c)
```

details.txt:
```
Hi
I
am
Dhanujaaaaaaa
```

O/P: 1

function to return lines with only lowercase characters

```
def func1():
    result=[]
    with open("details.txt","r") as f1:
        for i in f1:
            for j in i:
                if j.islower():
                    result.append(j)

    return result

c=func1()
print(c)
```

details.txt:
```
hello, Dhanu
```

O/P: ['h','e','l','l','o','h','a','n','u']

modules

import os

os.system, os.open both are used to executed system commands in windows/linux

os :: To interact with system os

os.system() will return status code

os.popen() will return command output

→
```
os_out = os.system("vol")
print("output of os.system", os_out) #Exit code
os_out = os.popen("vol")
print("output of os.popen", os_out.read())
                                # Actual output
```

os.listdir() :: To list files & folders

os.mkdir() :: To create new directory

os.rmdir() :: To remove directory

os.chdir() :: To change directory

→
```
print(os.listdir())
print(len(os.listdir()))
os.rmdir("sample_dir")
if "sample_dir" in os.listdir():
        print("File Exists")

else:
    print("creating File", os.mkdir("sample_dir"))
```

→
```
import os
os.mkdir("dir10")
print(os.getcwd()) # c:\users\DELL\Desktop\winteck
os.chdir("dir10")
print(os.getcwd()) # c:\users\DELL\Desktop\
                        winteck\dir10
```

→ import os
print(os.path.exists("dir1"))
print(os.path.realpath("dir1"))
print(os.path.abspath("dir1"))
print(os.path.isdir("dir1"))
print(os.path.isfile("os1.py"))

```
if os.path.exists("dir"):
    if os.path.isdir("dir1"):
        print("dir")
    else:
        print("File")
```

→ in cmd we have to run in notepad →
```
import os
print(os.curdir)
print(os.getcwd())
```

→ import os
→ os.curdir
'.' → It represents current directory in linux

→ os.getcwd()
'C:\\Users\\DELL\\Desktop\\wintech"

getcwd :: os.getcwd() :: To get current working directory

os.curdir :: '.' :: current directory

→ os.get_terminal_size()
os.terminal_size(columns=61, lines=32)

→ If we want to see environmental variables in cmd
↓
os.environ :: To get system environ variables

→ dir(os.path) :: To get all methods of path

→ import sys

sys :: used to interact with system and get / change system configuration

ex :: env variables

```
print(dir(sys))
print(sys.winver)    # python version
print(sys.platform) # platform details # win32
print(sys.version)   # python version
print(sys.getsizeof("a")) # to get size of a variable
```

→
```
x=[1,2,3]
y=(1,2,3)
print(sys.getsizeof(x)) # 80
print(sys.getsizeof(y)) # 64
```

→
```
import os
print(os.getcwd())
```

→ import sys

sys.path :: To get all python paths

sys.path.insert() # To add path to PYTHONPATH to search for modules

→ python will search for user defined modules in sys.path

→
```
sys.path.insert(0,"E:\\")
import module4
```

⇒
```
import sys
sys.path.insert(0,"C:\\")
print(sys.path)
```

⇒
```
import sys
data=sys.stdin.readline()
print("you entered:",data)
```

O/P: Hi
you entered: Hi

⇒
```
import sys
sys.stdout.write("Hello world\n")
```

O/P: Hello world

⇒
```
import sys
sys.stderr.write("this is an error message\n")
```

O/P: this is an error message

→ import sys

```
sys.exit()  # To stop python process / to exit python script
sys.exit(message)
sys.exit(1)

print ("line1")
sys.exit("Exit from python")
sys.exit(1)
print("line2")
```

→ import sys cmd: python sys4.py 10 20

```
print(sys.argv) # ['sys4.py', '10', '20']
print (sys.argv[0]) # sys4.py
print (sys.argv[1]) # 10
print (sys.argv[2]) # 20
```

→ cmd; python sys4.py 10 20

```
x = int (sys.argv[1])
y = int (sys.argv[2])  # 30
print (x+y)
```

Diff b/w sys.argv and input()

input() :: used to read user inputs in runtime

sys.argv() :: used to read command line arguments

→ import copy

```
copy.copy()      # shallow copy
copy.deepcopy()  # deep copy

list1 = [1,2,3,4]
list2 = copy.copy(list1)  # shallow copy

list3 = [1,2,3,4]
list4 = copy.deepcopy(list3)  # deep copy
```

```
list1.append(5)
list3.append(5)
print(list1)  # [1,2,3,4,5]
print(list2)  # [1,2,3,4]
print(list3)  # [1,2,3,4,5]
print(list4)  # [1,2,3,4]

if list1==list3:          #equal
    print("equal")

else:
    print("not equal")
```

```
import copy
list1=[1,2,[3,4,5],[6,7]]
list2=copy.copy(list1)
list1[2][0]=999

print(list1)  # [1,2,[999,4,5],[6,7]]
print(list2)  # [1,2,[999,4,5],[6,7]] → also changed

list3=[1,2,[3,4,5],[6,7]]
list4=copy.deepcopy(list3)

list3[2][0]=999

print(list3)  # [1,2,[999,4,5],[6,7]]
print(list4)  # [1,2,[3,4,5],[6,7]] → stays the same
```

Difference b/w shallow copy and deep copy

Shallow copy:

→ creates a new outer list
→ But inner objects are not duplicated - they are just referenced
→ This means changes to nested elements in the original will also affect the copy.

Deep copy:

→ creates a completely independent copy
→ All nested objects are duplicated
→ changes to the original will not affect the copy →Deep copy take more memory and slower because they duplicate everything

Difference b/w realpath and abspath

Both os.path.realpath() and os.path.abspath() are used to get the absolute path of a file (or) directory in python. But there is a key difference in how they handle symbolic links

for ex: /hom/user/data is symlink to /mnt/storage/data

```
import os
path = "/home/user/data"
print("abs path:", os.path.abspath(path))
print("real path:", os.path.realpath(path))
```

o/p: abspath: /home/user/data
real path: /mnt/storage/data

→ abspath(): gives the full path as it appears.
→ realpath(): follows the symbolic link and gives the true destination.

→ import time
print("Before sleep")
time.sleep(10)
print("After sleep")
print(time.ctime()) # Wed Aug 13 10:32:24 2025
→ current time

→ import datetime
date1 = datetime.datetime.now()
str1 = "2025-08-12 19:48:24.821244"
req_date = str1.split(" ")[0]
print(req_date)
print(date1.strftime("%d-%m-%Y")) # 13-08-2025

Subprocess::

To execute system commands windows, Linux, Power shell

→ It is used to interact with child process
→ we can send input to the subprocess.PIPE

```
import subprocess
subprocess.run("dir")
out = subprocess.popen("fdisk /dev/sda")
subprocess.call()
```

→ import subprocess
out = subprocess.popen(cmd, stdin=subprocess.PIPE,
stdout=subprocess.PIPE,
stderr=subprocess.PIPE)
#out.communicate("m")
out.close()

Regular Expressions:

→ Regular expressions are used to pattern matching in strings.

→ we can use regular expression in python by importing re module

→ import re

Why ?

→ To search / Find / Match / Replace / Extract data from strings

re.search;;

⇒ import re

re.search() # To search for a keyword in entire string

Str1 = "Microsoft Windows [version 10.0.22000.1817]"

syntax ::

re.search (Pattern / keyword, string)

```
if re.search ("windows", Str1):
    print ("Match found")
if re.search("windows", Str1, re.I):
    print ("Match found")
```

re.I = Ignore case

⇒ import re

Grouping :: To capture / Extract the data from string

```
Str1 = "Microsoft Windows [version 10.0.22000.1817]"
search_out = re.search ("(windows)", Str1, re.I)
print (search_out.groups()) # ('windows',)
Print (search_out.group(0))
print (search_out.group(1))
```

```
impat re
str1 = "python version 3.0"
search_out = re.search ("(python) (version) (3.0)" str1)
print(search_out.groups())
                    # ('python', 'version', '3.0')
```

→ Strftime:

It means string format time
It takes a datetime object and converts it into a string in the format you choose.

Ex:

```
import datetime
date1 = datetime.datetime.now()
print(date1)    # 2025-08-13 10:58:31.026741
print (date1.strftime("%d-%m-%Y")# 13-08-2025
print (date1.strftime ("%Y-%m-%d")#2025-08-13
```

→ Strptime:

It means string parse time
It takes a date/time string and converts it into a datetime object according to given format

Ex:

```
import datetime
date1 = "2025-05-20"
print (datetime.datetime.strptime
        (date1, "%Y-%m-%d"))
        # 2025-05-20 00:00:00

print date2 = "13-05-2025 12:35"
print(datetime.datetime.strptime(date2,
        "%d-%m-%Y %H:%M"))
        # 2025-05-13 12:35:00
```

Date formatting:

→ It means converting a date/time into a specific human-readable string using format codes.

format codes::		EX::
%.d	- Day of the month (01-31)	- 13
%.m	- Month number (01-12)	- 08
%.b	- Month name, short	- Aug
%.B	- Month name, full	- August
%.y	- year (2 digit)	- 25
%.Y	- year (4 digit)	- 2025
%.H	- Hour (00-23)	- 14
%.I	- Hour (01-12)	- 02
%.p	- AM/PM	- PM
%.M	- Minutes (00-59)	- 45
%.S	- seconds (00-59)	- 07
%.a	- weekday name, short	- wed
%.A	- week day name, full	- wednesday
%.j	- Day of the year (001-366)	- 225

re.match:: To match from beginning of the string

```
import re
str1 = "python version 3.0"
re.match ("version 3.0", str1) # False
re.match ("python version 3.0", str1) # True
```

re.findall() :: To capture all matched keywords in a list format.

```
import re
str1= '''Error

Error
Error
pass
Fail
Fail
Pass'''
out1 = re.findall ("Error", str1)
print (out1)  # ['Error', 'Error', 'Error']
```

Ex::

* 10000 lines in a file, we have to search for keyword "Error", print line and line number

```
with open ("file1", "r") as f1:
    lines = f1.readlines ()
    for i,j in enumerate (lines):
        if re.search ("error", j):
            print (j, i)
```

→ . :: Any character except newline.

^ :: Start of String

$ :: End of String

* :: 0 or more repetitions

+ :: 1 or more repetitions

? :: 0 or 1 repetition

{n} :: Exactly n repetitions

{n,} :: n or more repetitions

{n,m} :: Between n and m repetitions

( ) :: Group
\ :: Escape special character or indicate special sequence
\d :: Digit [0-9]
\D :: Non-digit
\w :: word character [a-zA-Z0-9_]
\W :: Non-word character
\s :: whitespace
\S :: Non-whitespace

Ex:-

Employee Name is emp(\d+) and ID is (\d+{3})

result=[]
for i in data:
    if

Ex::

```
import re
str1 = "1234"
re.search ("\d", str1)      # 1
re.search ("\d+", str1)     # 1234
re.search ("\d?", str1)     # 1

str1 = "1234 python"
search_obj = re.search("(\d+)\spython", str1)
print (search_obj.group(1))
```

Ex:: str1 = "1234 1234"
pattern = "\d+\s\d+"

{} :: Exact matchings
pattern = "\d{4}\s\d{4}"
mn = "8091919191"
pattern = "\d+"
pattern = "\d{10}"

Ex:: import re
str1 = "python 312"
# pattern = "\w+\s\d+"
str1 = "python version is 3.0"
pattern = "python version is (.*)"
search_obj = re.search(pattern, str1)
print (search_obj.group(1))

Ex:: import re
str1 = "Linux Kernal is RHEL and version is 9.2"
Pattern = "Linux Kernal is (\w+) and version is (\d+.\d)"
Pattern = "\w+\s\w+\s\w+\s(\w+)\s\w+\s\w+\s\w+(\d+.\d)"
obj = re.search(pattern, str1)
Kernal = obj1.group(1)

```
version = obj1.group(2)
print(kernal, version)
```

Ex::
```
import re
str1 = "12345abcd"
print(re.findall("\d", str1))
print(re.findall("[a-z]", str1))
```

Difference b/w re.match() and re.search()

re.match() ::

→ check only at the beginning of the string
→ Returns a match object if pattern is found at position 0
→ Returns None if the match is not at the start

re.search() ::

→ searches anywhere in the string for the first match
→ Returns the first match object it finds
→ Returns None if no match is found

⇒
```
import re
str1 = "computer Name is DESKTOP-SH8KA33"
out = re.search("\w+\S\w+\S\w+\S(\w+-\w+)", str1).group(1)
print(out)
str1 = "1234 abc 456 def 7895 12 34"
print(re.findall("\d{3,4}", str1))
```

⇒ # {a,b} → a to b times

```
import re
str1 = " python version 3.0"

# ^ :: start of string
# $ :: end of string
pattern = " ^python. .* "
print (re. match (pattern, str1))
pattern = "^python .*0$ "
print (re.match (pattern, str1))
```

```
import re
str1 = " 7012345679"
pattern = "^[6-9] {1} \d {9} $"
obj = re.match (pattern, str1)
print (obj)
```

6 to 9, total 10.
No character, space, special character

```
import re
str1 = "Linux"
pattern = "python | linux"
print (re.search (pattern, str1))
```

```
import re
str1 = "123 abc"
str2 = "12 abc"
pattern = "\d {2} | \d {3}"
print (re. search (pattern, str1))
```

#match only special characters

```
import re
str1 = "@#$"
Print(re.match (" \W+ ", str1))
```

→ str1 = "winteck1234@gmail.com"
str2 = "winteck_123@gmail.com"

re.match("\w+\d+@gmail.com")

✓ re.match("[a-z]+\W*\d+@gmail.com")

re.match("^[a-zA-Z]+\w+\W+@gmail.com")

✗

→ ip1 = "192.168.0.112"
ip2 = "10.123.1.1"
✗ ip3 = "100.0.0.1"
ip4 = "192.168.1233.1"
ip5 = "10.111.257.2"
ip6 = "100.200"

re.match("\d{2,3}\.

pattern = "^[a-z]+\d+@gmail.com"

✗

```
#function should take vm_details as argument
#return dictionary
# {"VM1":"centos", "VM2":"Ubuntu", "VM3":"RHEL9",
   "VM4":"windows8"}

import re
def get_vm_details(vm_details):
    vm_details1={}
    for line in vm_details.split("\n"):
        search_obj=re.search("(VM[0-9])\S*:\S*(\w+)",
                             line)
        if search_obj:
            vm_name = search_obj.group(1)
            OS_type = search_obj.group(2)
            vm_details1[vm_name]=OS_type
    return vm_details1
vm_details=get_vm_details(vm_details)
print(vm_details)
```

```
import re
Str1 = """ Drive1 :: /dev/sda
Drive2 :: /dev/sdb
Drive3 :: /dev/sdc """
def get_drive_list():
    drive_list = re.findall ("Drive\d{1,2}\s::\s
                 (\/dev\/sd[a-z])", Str1)
    return drive_list
drive_list = get_drive_list()

Str2 = "OS Disk is /dev/sda"
res = re.search ("OS Disk is (/dev\/sd[a-z])", Str2)
out = res.group(1)

if out in drive_list:
    drive_list.remove(out)

print(drive_list)
```

O/p: ['/dev/sdb', '/dev/sdc']

Lamda: Lamda functions are anonymous function -s in python which are defined in a single line

→ Lamda function will take multiple arguments and single expression

Syntax:: fun_out = lamda arguments : expression

Ex:: →add = lambda x,y : x+y
print (add (10,20))

→ fun_out = lamda x : x**2
Print (fun_out (2))

→ l1 = ['a', 'b', 'c']
fun1 = lambda x : x+'%'
for i in l1:
    print (fun1 (i))

→ even = lambda x: x%2==0
print(even(10))

filter: filter is a in-built function in python which is used to filter the sequence by applying some function

syntax :: filter(function, sequence)

→ filter will return the filter object we need to convert filter object to required format

Ex:: output = (filter(lambda x: x%2==0, [1,2,3,4,5]))
print (list(output)) #[2,4]

Ex:: list1 = [10,20,30,40,50]
~~out2= lambda x: x>30~~
output = list(filter(lambda x: x>30, list1))
print (output)

Ex:: ~~#~~ import re
l2 = ['12 34', 'str1', '1234', '12&34']
~~pattern = \S\W~~ ~~\d{2}\S\d{2}\d{2}\W\d{2}~~
print(list(filter(lambda x: re.search('\S|\W', x), list1)))

Ex:: import re
list1 = ['abc', '123abc', '1234']
print [list(filter(lambda x: re.search('\d', x), list1))]
print(list(filter(lambda x: not x.isalpha(), list1)))

map: map will map the function to a sequence and returns the map object.
→ we need to convert map object to a required format.

Syntax: map(function, sequence)

Ex: print(map(lambda x: x**2, [1,2,3,4]))

Ex: l1 = ['a','b','c','d']
print(list(map(lambda x: x+'*', l1)))

→ l2 = ['sda','sdb','sdc']

print(list(map(lambda x: '/dev/'+x, l2))

reduce: Reduce will reduce the sequence to a single element

→ we need to import reduce from python module.
@functools

→ from functools import reduce

Ex: list1 = [1,2,3,4]
print(reduce(lambda x,y: x+y, list1))

Errors:

→ Attribute error, Value error, syntax Error, Indentation error, Type Error, Name Error

Exception handling:

→ used to handle exceptions in python

Errors: [some issues in python code] that

→ from python code, ~~which~~ we can't handle

Ex: Syntax error, Indentation error

Exceptions: Runtime errors from python script

→ we can handle the exceptions using try & except blocks

Ex: Name Error, zero division Error, fileNotFoundErro, Key Error

→ the main block of code we need to write inside try block:

If any error in try block we can catch it using except block and handle it.

Syntax: try:
<Main code>
except Exception as err: #err=error from try block
print(err)

→ when we define try block, except block is also mandatory

→ we can define multiple except block for single try block.

Ex: #No Error

→
```
x=100
try:
    print(x)
except Exception as err:
    print("x value is not defined")
```

#Syntax Error we can't handle it

→
```
try:
    print("Hello")
except Exception as err:
    print(err)
```

ZeroDivisionError we can handle it
Except block will execute

→
```
try:
    print(10/0)
except Exception as err:
    print(err)
```

Ex:
```
try:
    print(10/0)                    #Error
except NameError:
    print("In Except Block")
except ZeroDivisionError:
    print("ZeroDivision Error")
print("Hello world")
```

Ex:
```
print(10/0)
print("Hello world")
```

→ Ex:
```
x=10
#To raise our own exceptions
#raise exception("message")
if x>100:
    raise Exception("The value of x should be >100")
    # print("The value of x should be >100")
else:
    print(x)
print("After Error")
```

Ex:
```
try:
    print(10/0)
except exception as Err:
    raise Exception("exception from try block")
    #catch exception from try block and handling
    it in except block
```

→
```
Path=""
try:
    with open(Path,"r") as f1:
        data = f1.read()
except fileNotFoundError:
    Print("Path not found Please provide valid Path")
    Path = input("Enter valid Path")
```

```
with open(path, "r") as fl:
    data = fl.read()
print(data)
```

→ list1 = [10,40,19,5,49,10,49]
second largest number by removing duplicates

```
list1 = [10,40,19,5,49,10,49]
uniq_list = set(list1)
if i in uniq_list:
    if i>
```

dict1 = {"a":10,"b":20}
New dictionary by swapping keys & values.

```
swapp_dict = {i:j for j,i in dict1.items()}
Print (swapp_dict)
```

Open a file and count no.of digits, Alphabets and spaces.

```
with open ("file1.txt", "r") as file2:
    text = file2.read()

list1 = [ ]
if i.isdigit
  for i in text:
```

→ str1 = "aaabbaccac"
print longest repeated character

→ count occurence of each character
str1 = "aabbccddeee"
result = {}
for i in str1:
    result[i] = result.get(i,0)+1
print(result) {"a":2, "b":2, "c":2, "d":2, "e":3}

→ create a new string by removing consecutive duplicate elements

str1 = "aabbccddeee"
~~new str1 = str1[0]~~
str2 = ""
for i in str1:
    if i not in str2:
        str2 = str2 + i
print(str2)

---

ex::

```
try:
    print(10/0)
except ZeroDivisionError:
    print("If Zero Division Error")
except KeyError:
    print("If key error")
finally:
    print("In finally block")
```

finally Block::

→ irrespective of the result from try and except block, code in finally block will get executed

else:

try:
→ If there is no error in try block the code will else block will get executed and else block is optional

Ex:
```
try:
    print("No error")
except Exception as err:
    print("In exception")
else:
    print("In else")
finally:
    print("In finally block")
```

→ If we implemented try block either except (or) finally is mandatory
→ we can write multiple except blocks for a single try block.

```
# To raise own exceptions in python
x=100
if x!=10:
    raise Exception("Error")  #sys.exit(1)
    #terminates python execution
```

Oops:

→ python supports
① Procedure oriented programming
② function " "
③ object " "

oops (object oriented programming):
→ In object oriented programming we use classes & object instead of just using functions

Advantages:

→ code reusability
→ security
→ we can solve real-world problems
→ easy to maintain

Features:

→ class
→ object
→ Inheritance
→ Encapsulation
→ polymorphism
→ Abstraction

class:: class is a blueprint for creating objects
→ we can define the class by using a keyword "class"

syntax:: class class_name:
    #attributes & methods

→ class contains attributes & methods
→ Attributes = variables
   methods = functions (outside the class)

```
Ex:: class cls1:
        x=100  # attributes
        def method1 (self): # Method
            print("In method1")
```

Object :: object is an instance of a class. By using object we can access properties of the class (attributes & methods)

Ex:: we need to create an object for the class and we can create "n" no.of objects for a single class

obj1 = cls1() → creating an object
print(obj1.x) → Access attribute
Obj1.method1() → Access method

Self: self represents the current object name
→ self is not a keyword we can pass any other name as a first argument of a method and it is mandatory
→ By using self we can access the other properties of class
→ self is mandatory in instance methods.
ex:: class cls1:
    x = 100 # attributes
    def method1 (self): # method
        print (self.x)
        print ("In method1")

obj1 = cls1()
obj2 = cls1()
print(obj1.x)
obj1.method1()
obj2.method1()

→ obj1 = cls1()
print(type(obj1)) # <class '__main__.cls1'>
x = 100
print(type(x)) # <class 'int'>

constructor:

→ __init__() is a special method in python it is used as a constructor which is used to initialize all the attributes
→ __init__() is an instance method and first argument should be self

Ex:

```
class cls1:
    def __init__(self, a, b):  # instance method
        print("Inside init")
        self.a = a
        self.b = b
        print(self.a, self.b)
obj = cls1(100, 200)          #100 200
# All the __init__() arguments must be passed while creating an object.
```

Ex::

```
class cls1:
    def __init__(self, a, b):
        self.a = a
        self.b = b
    def method1(self, c):
        print(self.a)
        print(self.b)
        print(c)
obj2 = cls1(100, 200)          #100
obj2.method1(300)              200
                               300
```

class variables:: will be defined inside the class and outside the method

→ To access class variables in instance methods

1. self.class_var
2. classname.class_var

Instance variables:: will be defined inside the instance method

→ To access instance variables in instance methods

1. self.var_name

Ex:

```
class cls1:
    class_var = 100  #class variable
    def __init__(self, a, b):
        self.a = a  #instance variable
```

```
        self.b=b
    def method1(self):
        print(self.a)
        print(self.class_var)
        print(self.cls1.class_var)

obj1=cls1(200,300)                    # 200
obj1.method1()                        #  100
                                      #  100
# To access/modify class variables outside the class
obj1=cls1(200,300)
print(cls1.class_var)                 # 100
obj1.class_var=1000                   # 1000
print(obj1.class_var)
# To access instance variables outside the class
print(obj1.a)                         # 200
```

Inheritance:: Inheriting the properties of parent class in the child class. They are diff types of inheritance

1. Single level inheritance
2. Multi level inheritance
3. Multiple inheritance

1. Single level inheritance:: In single level inheritance we will inherit the properties of single parent class in child class

```
ex: class cls1:
        def method1(self):
            print("In cls1 and method1")
    class cls2(cls1):  # single level
        def method2(self):
            print("In cls2 and method2")

obj1=cls2()
obj1.method1()        #In cls1 and method1
obj1.method2()        #In cls2 and method2
```

#with child class object we can access both parent

class & child class properties

Ex:
```
class cls1:
    def __init__(self,a):
        self.a=a
    def method1(self):
        print(self.a)
        print("In method1")

class cls2(cls1):
    def __init__(self,a,b):
        self.b=b
    def method2(self):
        print(self.b)

obj1 = cls2(
```

```
class cls1:
    def __init__(self,a):
        self.a=a
    def method1(self):
        print(self.a)

class cls2(cls1):
    def __init__(self,a,b):
        super().__init__(a)
        self.b=b

obj1=cls2(100,200)
obj1.method1()
```

#100

super():: used to inherit parent class constructor in child class

→ we can use super() or class name to inherit parent class constructor in child class

Multi-level inheritance:: In multi-level inheritance we will inherit the parent class properties in child class and child class properties in another child class

→ It is a chain of inheritance

Ex::

```
class cls1:
    def __init__(self,a):
        self.a = a
    def method1(self):
        print(self.a)
class cls2(cls1):
    def __init__(self,a,b):
        super().__init__(a)
        self.b=b
    def method2(self):
        print(self.b)
class cls3(cls2):
    def __init__(self,a,b,c):
        super().__init__(a,b)
        self.c=c
    def method3(self):
        print(self.c)
obj3=cls3(10,20,30)
obj3.method1()
obj3.method2()
obj3.method3()
```

#100
200
300

→ By using the object of cls3 we can call or access methods and attributes from cls1 & cls2

Multiple inheritance:: In multiple inheritance we can Inherit multiple parent classes in the child class.

Ex::

```
class cls1:
    def __init__(self,arg1):
        self.arg1 = arg1
    def method1(self):
        return self.arg1
class cls2:
    def __init__(self,arg2):
        self.arg2=arg2
```

```
    def method2(self):
        return self.arg2

class cls3(cls1, cls2):
    def __init__(self, arg1, arg2, arg3):
        super().__init__(arg1)
        cls2().__init__(self, arg2)
        self.arg3 = arg3
    def method3(self):
        return self.arg3

obj1 = cls3("a","b","c")
print(obj1.method3())   # c
print(obj1.method2())   # b
print(obj1.method1())   # a
```

Ex:

```
class c1:
    def method1(self):
        print("In c1 and method1")

class c2:
    def method1(self):
        print("In c2 and method2")

class c3(c2,c1):
    def method3(self):
        Print("In c3 and method3")

obj3 = c3()
obj3.method1()   # In c1 and method1
```

MRO (Method Resolution order):: It defines how python will look for methods incase of inheritance

Ex: class c1: → In single level inheritance MRO will be child class → Parent class

→ In multiple inheritance child class

```
class c1:
    pass
class c2:
    pass
class c3(c1,c2):
```

MRO will be c3 → c1 → c2

→ To get MRO of a class
classname.mro ⟹ print(C3.mro()) as a function.

```
class cls1(x):
    def __init__(self,a,b):
        self.a=a
        self.b=b
    def method1(self):
        return print(self.a + self.b)

class cls2(x):
    def __init__(self,a,b,c,d):
        super().__init__(a,b)
        self.c=c
        self.d=d
    def method2(self):
        return (self.c+self.d)

obj1 = cls1(4,5)
obj2 = cls2(5,4)
obj1.method1()
obj2.method2()
```

```
def compare(cls1, cls2):
    if cls1 > cls2:
        print(obj1)
    else:
```

---

① Public Attribute :: Public attribute will be available across the class and we can access/modify it outside the class also.

ex::
```
class cls1:
    def __init__(self,a,b,c):
        self.a=a   # public Attribute
        self.b=b
        self.c=c
    def method1(self):
        print(self.b)
class cls2(cls1):
    def method2(self):
        print(self.a)

obj1=cls1(10,20,30)
print(obj1.a)
print(obj1.b) print(obj.c)
```

② protected Attributes:: It shouldn't be accessed directly outside the class

→ protected Attributes will be initialized with the prefix of -("single underscore")

Ex:: class cls1:
```
    def __init__(self,a,b,c):
        self.a=a
        self._b=b  # protected Attribute
        self.c=c
class cls2(cls1):
    def __init__(self,a,b,c):
        super().__init__(a,b,c)
    def method1(self):
        return self._b

obj1=cls1(10,20,30)
print(obj1.method1())
```

③ private Attribute:: we cannot access private attributes outside the class

→ these are accessible within the parent class only

Ex:: → It will be defined with a prefix of --("double underscore")

→ we can still access private attributes outside the class by using ~~name~~ "name mangling"

Ex:: class cls1:
```
    def __init__(self,a,b,c):
        self.a=a    # public
        self._b=b   # protected
        self.__c=c  # private
obj1=cls1(10,20,30)
print(obj1._b)
# print(obj1.__c)
```
Syntax:: object._classname__attributename
```
print(obj1._cls1__c)
```

```
try:
    print(obj1)
except Exception as err:
    pr
except AttributeError:
    print(obj1.cls1.c)
```

Encapsulation:: Encapsulation is a process of making the attributes in private and restricting the direct access to attributes and it is a process of binding the data(attributes & methods) into a single class
→ we can access the attributes by using getter and setter methods

Ex::
```
class cls1:
    def __init__(self,a,b):
        self.a=a
        self.__b=b
        self.__c=0
    def setter(self,c):
        self.__c=c
    def getter(self):
        return self.__c
obj1=cls1(10,20)
obj1.setter(100)
c=obj1.getter()
print(c)
```
100

class Method:: class methods are created to access/modify class level attribute

Types of methods::
① Instance method ② class method ③ static method

① **Instance method::** By using instance method we can access both class level attribute and instance attribute
→ It is having all the advantage of OOPS

Ex:
```
class cls1:
    a=10
    def __init__(self, arg1):
        self.arg1=arg1
    def method2(self):
        pass
    def method1(self):  # instance method
        self.method2()
        return self.arg1

obj1=cls1(100)
```

② class method :: class methods are created to access/modify only class level attribute
→ first argument of class method should be "cls" → class name
→ we need to decorate method with @ classmethod decorator
→ we can't access instance attributes with "cls"

Ex:
```
class cls1:
    a=10
    def __init__(self, arg1):
        self.arg1=arg1

    @classmethod
    def method1(cls):
        cls.a=100

obj1=cls1(10000)
obj1.method1()
print(obj1.a)
```

③ static method :: If we are not going to access/modify class (or) instance attributes (or) methods we can make method as a static method
→ static methods are independent of classes
→ we need to decorate a method with @staticmethod
→ static methods doesn't require any arguments

ADVANTAGES::
→ No dependency on classes & objects
→ used as a standalone functions within the class
→ used to implement helper functions

Ex:: class cls1;

```
    a=10
    def __init__(self, arg1):
        self.arg1=arg1
    def instance_method(self):
        return self.arg1

    @staticmethod
    def static_method():
        print("standalone method")

obj1=cls1(100)
obj1.instance_method()
obj1.static_method()        #standalone method
```

Polymorphism:: python by default supports polymorphism
→ polymorphism means many forms
→ In python oops it allows same method to behave differently based on input
→ we can achieve polymorphism in python in different ways ① By using inbuid functions
② Method overloading
③ Method overriding

① polymorphism by using inbuild functions::

Ex: len, max, min
→ len([10,20]) → max(1,2,3) → min(1,2,3)
len("abc") → max("a","b","c") → min("a","b"
len({1,2,3,4,5,6})

② method overloading:: By default python won't support method overloading
→ we can overload the methods by using variable length positional arguments (*args)

Ex:: class cls1:
    def method1(*args):
        print(args)

obj1=cls1()
obj1.method1(100)
obj1.method1(100,200,300)
obj1.method1("a","b")

③ Method overriding:: If parent class and child class having same methods and if we impliment inheritance the method in the parent class will be override in the child class

Ex:: class cls1:
    def method1(self):
        print("cls1.method1")
class cls2(cls1):
    def method1(self):
        print("cls2.method1")

obj1=cls2()
obj1.method1()

for multilevel inheritance with method overriding

class A:
    def __init__(self,a,b):
        self.a=a
        self.b=b
    def method1(self):
        print(self.a+self.b)

class B(A):
    def __init__(self,a,b,c,d):
        super().__init__(a,b)
        self.c=c
        self.d=d
    def method1(self):
        print(self.c+self.d)

class C(B):

```
def __init__(self,a,b,c,d,e,f):
        super().__init__(a,b,c,d)
        self.e=e
        self.f=f
    def method1(self):
        print(self.e * self.f)

obj1 = cls1(1,2,3,4,5,6)
obj1.method1()      # 30
obj1.method1()      # 30
```

Abstraction :: Abstraction means hiding the internal details of class and showing only required features

→ In python we can achieve abstraction by using abstract class and abstract method.

Ex:

→ If we want to create a class & method and implement it later we can make the classes abstract class

→ If we create a abstract class we need atleast one abstract method

Ex1: → In python abstraction is not directly supported we need to import a class called ABC and abstract method from abc module

abc = Abstract Base class

Ex:

```
from abc import ABC, abstractmethod
class cls1(ABC):
    def show_details(self):
        pass
    @abstractmethod
    def functionality(self):
        pass
class cls2(cls1):
    def functionality(self):
        print("functionality of cls1")

obj1 = cls2()
obj1.functionality()
```

Ex2:: To create an empty class & method

```
from abc import ABC, abstractmethod
class, cls1 (ABC):
    @abstractmethod
    def method1 (self):
        pass

obj1 = cls1()
obj1.method1()
```

Ex3:: from abc import ABC, abstractmethod

```
class vehicle(ABC):
    @abstractmethod
    def show_details():
        pass
class car(vehicle):
    def show_details (self):
        print("These are car details")

obj1 = car()
obj1.show_details()
```

PEP8 Rules::

→ Should not mix spaces & tabs
→ use doc strings for classes/methods and functions
→ use one line gap b/w the functions
→ Don't use uppercase characters in function/method names
→ Don't use special characters in variable names
→ follow standard import order
    1. python inbuilt
    2. user installed
    3. user defined
→ maximum length of a line should be 79
→ line gap b/w two functions/classes
→ Don't use white spaces in function calls
→ Meaningful names for classes, functions, variable & methods

→ Don't use Python inbuilt function names as variables/function/method names

→ Give one empty line at end of code

In python we can check coding standards by using Pylint

Ex:: Pylint <script_name>

---

obj)d{2}+:\d{2}3+:d{2} up ("\d{2}:\d{2}").group(1)

re.search(obj, group(1)

~~import pa~~

paramiko::

~~import paramiko~~

→ paramiko module in pythod is used to communicat with remote servers with SSH protocol

→ we can execute commands on the remote servers and get the response using paramiko

Ex:: import paramiko

```
username = "winteck"
password = "winteck@2025"
ip = "192.168.0.103"
port = 22
new_hostname = "winteck1"
def exec_cmd(command):
    try:
        ssh_obj = paramiko.SSHclient()
        ssh_obj.set_missing_host_key_policy(parami
            -ko.AutoAddPolicy())
        ssh_obj.connect(ip, port, username, password)
        stdin, stdout, stderr = ssh_obj.exec_command
                                (command)
        stderr = stderr.read().decode()
```

```
        if stderr:
            raise Exception("command execution failed")
        return stdout.read().decode()
    except Exception as err:
        raise Exception(err)

uname = exec_cmd("uname")
print(uname)
```

Threading: used to run multiple threads parallely and achieve multithreading in python

```
import threading
import time
def task1():
    for i in range(1,3):
        print(f"TASK 1 - iteration {i}")
        time.sleep(1)

def task2():
    for i in range(1,3):
        print(f"TASK 2 - iteration {i}")
        time.sleep(1)

#creating thread
thread1 = threading.Thread(target=task1)
thread2 = threading.Thread(target=task2)
# starting thread
thread1.start()
thread2.start()
# joining threads
thread1.join()
thread2.join()
```

```
# task1 - iteration1
  task2 - iteration1
  task2 - iteration2
  task1 - iteration2
```

Logging ::

```
# importing module
import logging
# To push msgs to a log file
# create and configure logger
logging.basicConfig(filename = "newfile.log", format=
        '%(asctime)s %(levelname)s %(message)s',
        filemode='w')
#creating an object
log = logging.getLogger()
#setting the threshold of logger to Debug
log.setLevel(logging.DEBUG)
#Test messages
log.debug("debug message")
log.info("Just an information")
log.warning("Its a warning")
log.error("Error message")
log.critical("Critical Error")
```

python threadl.py
newfile.log (file ex)

```
2025-10-8 12:13:50,135 DEBUG debug
2025-10-8 12:13:50,135 INFO just an in
2025-10-8 12:13:50,135 WARNING its a war
2025-10-8 12:13:50,135 ERROR error mes
2025-10-8 12:13:50,135 CRITICAL critical
```

Argparse ::

To read command line arguments in python

```
#To create an ArgumentParser object
import argparse
parser = argparse.ArgumentParser(description="Description")
#parsing arguments
parser.add_argument("drive", help="Give drive name in
                    string format")
parser.add_argument("--timeout", default=30, help=
                    "Mention timeout")

#parse the arguments
args = parser.parse_args()
print(f"your drive is: {args.drive}")
print(f"timeout is {args.timeout}")
```

Decorator:

→ Decorator is a special type of function in Python

→ Decorator is a basically a function inside the function which takes function name as arguments

→ We can decorate normal function with decorator function so that it will add extra functionalities to the existing function without modifying its source code

→ In Python we can decorate function with @ decorater name.

Ex:: def decorator (function):
    def fun2():
        print("Before fun1")
        function()
        print("after fun1")
    return fun2

@decorator
def fun1():
    print("Inside fun1")

fun1()

Ex:: from datetime import datetime
import time

Decorator function to calculate time taken by a original function

def decorator(function):
    def fun2():
        start_time = datetime.now()
        function()
        end_time = datetime.now()
        print(f"Time taken by function {end_time - start_time}")
    return fun2

@decorator

```
def fun1():
    for i in range(10):
        print(i)
        time.sleep(2)

fun1()
```

Generator::

→ It is used to generate sequence

→ It works based on "yield" Keyword and it return iterator

→ Generator is a special function in python

→ yield will capture the state of an object and iterable and it is memory efficient.

→ Generator function will return generator object

→ It we want to access individual iterable we can use next(object / function)

→ we can iterate through the generator object to print the sequence

Ex:1::
```
def fun1():
    for i in range(10):
        yield i

t1=fun1()
print(t1)
print(next(t1))
print(next(t1))
```

Ex:2::
```
def fun2():
    return 1
    return 2
    return 3
print(fun2())
```

```
def fun2():
    yield 1
    yield 2
    yield 3

print(fun2())
for i in fun2():
    print(i)
```

Ex:

```
class c1:
    def method1(self):
        print("class1")

class c2(c1):
    def method2(self):
        print("class2")

obj1 = c2()
obj1.method1()  #class1
obj2.method2()      class2
```

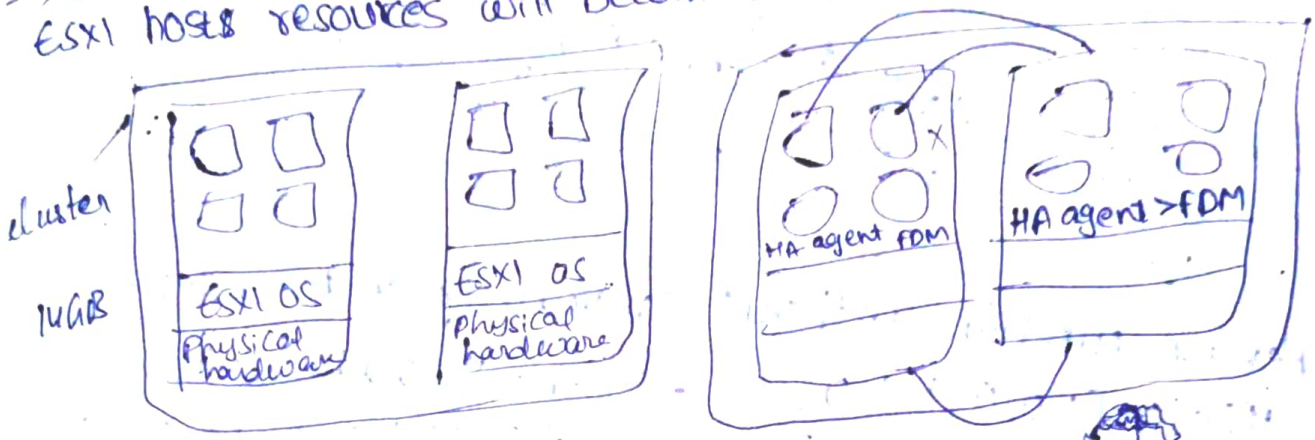
```
class c1:
    def __init__(self,a):
        self.a = a
    def method1(self):
        print(self.a)

class c2(c1):
    def __init__(self,a,b):
```

Decorator, generator, logging, argparse, threading, Paramiko
PEP8 rules, OOPs: class, object

→ Decorator
→ generator
→ return vs yield
→ logging
→ argparse
→ threading
→ PEP8 rules
→ oops (class) object, inheritance (single, multi, multiple), encapsulation, polymorphism (method overloading, method overriding), Abstraction
→ self, --init--
→ class variable, instance variable
→ super()
→ MRO
→ Public, Protected, Private
→ Types of methods (Instance, class, static)
→ Error / Exception / Exception handling
→ lambda, reduce, map, filter

Vcentre/vsphere:: It is a application from vmware used to manage multiple ESXI hosts
→we need to deploy vcentre VM on any of the hosts
steps to install vcentre::
~~half page (or) full page gap~~

cluster:: Group of ESXI hosts
→when we create clusters by adding multiple ESXI hosts the ESXI hosts resources will become part of cluster resource



Datastore::
VMFS file system
VM files
•VMX:: configuration files
•vmdk:: Storage files / Datastore files
•vmsn:: snapshot file
•vmware.log:: All VM log
vmkernal.log:: All Kernal logs
nvram.log:: SEL log

HA[High availability]:: HA stands for high availability whenever the ESXI host fails, the VM's which are running on ESXI host will be restarted in another ESXI host based on resource availability. ① HA should be enabled in cluster level ② when HA is enabled in cluster level, vcentre installs the HA agent called as FDM [Fault domain manager] on all ESXI hosts within the cluster ③ these HA agents [FDM's] will be in continuous sync with each other and exchanges heart beats ④ when one HA agent stops sending the heart beat other HA agent understands that other ESXI host got failed and starts restarting VM's on working ESXI host ⑤ There will be downtime in HA as VM's need to restarted on other ESXI host Requirements:: There should be enough resources in cluster ② HA must be enabled in cluster level ③ Atleast 2 ESXI hosts must be there in cluster disadvantage:: ① There will be downtime for VM's